A Project Report on

HOSTELEASE

A Digital Hostel Management Platform Using MERN

Submitted in partial fulfillment for award of

Bachelor of Technology

Degree

in

Computer Science and Engineering

By

Y. Mouli Reddy (Y22ACS594) **P. Pavan Kumar (Y22ACS538)**

K. Ajay Kumar (Y22ACS484) **V. Sai Ganesh (Y22ACS590)**

Under the guidance of

Ms. Deepika Kummari, M.Tech

Assistant Professor

Department of Computer Science and Engineering

Bapatla Engineering College (Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA – 522102, Andhra Pradesh, INDIA

2025-2026

Department of

Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled **HostelEase: A Digital Hostel Management Platform Using MERN** that is being submitted by Y. Mouli Reddy (Y22ACS594), P. Pavan Kumar(Y22ACS538), K. Ajay Kumar(Y22ACS484), V. Sai Ganesh(Y22ACS590) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide	**Signature of the HOD**
Ms. Deepika Kummari	**Dr. M. Rajesh Babu**
Assistant Professor	**Assoc. Prof. & Head**

External Signature

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

Y. Mouli Reddy (Y22ACS594)

P. Pavan Kumar (Y22ACS538)

K. Ajay Kumar (Y22ACS484)

V. Sai Ganesh (Y22ACS590)

Acknowledgement

We sincerely thank the following distinguished personalities who have offered their valuable advice and support for the successful completion of this work.

We are deeply indebted to our most respected guide, **Ms. K. Deepika, Assistant Professor**, Department of Computer Science and Engineering, for her valuable guidance, constant encouragement, constructive suggestions, and inspiring support throughout the course of this project.

We extend our sincere thanks to **Dr. M. Rajesh Babu, Associate Professor & Head of the Department**, Department of Computer Science and Engineering, for his cooperation and for providing the necessary facilities and resources to carry out this work successfully.

We would also like to express our heartfelt gratitude to our beloved **Principal, Dr. B. Srinivasarao**, for providing the required infrastructure, online resources, and institutional support to complete this project.

We express our sincere thanks to our **Project Coordinators, Dr. C. Prakasa Rao, Professor, Department of CSE, and Dr. R. Venkata Lakshmi, Assistant Professor, Department of CSE**, for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our **parents and family members** for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

Finally, we extend our sincere gratitude to all the teaching faculty and non-teaching staff of the Department of Computer Science and Engineering who helped us directly or indirectly with their cooperation and encouragement.

Y. Mouli Reddy (Y22ACS594)

P. Pavan Kumar (Y22ACS538)

K. Ajay Kumar (Y22ACS484)

V. Sai Ganesh (Y22ACS590)

ABSTRACT

HostelEase is a production-grade, full-stack web and mobile Hostel Management System (HMS) designed to modernize and streamline hostel administration operations in academic institutions. Built on the MERN stack (MongoDB, Express.js, React.js with Vite, and Node.js), HostelEase provides an integrated digital platform that eliminates manual, paper-based hostel workflows by offering a role-based, real-time, and centralized management environment.

The system supports four distinct user roles — Admin, Warden, Student, and Gatekeeper — each with dedicated dashboards and access-controlled functionalities. Core modules include student registration and profile management, room creation and bed allocation, leave request submission and multi-level approvals, maintenance issue reporting and resolution, real-time in-app notifications, analytics dashboards for wardens, comprehensive audit logging for accountability, and a dedicated Gatekeeper Portal for physical exit management.

Key technical features include JWT-based authentication with bcrypt password hashing and secure password reset via tokenized email links; role-based access control (RBAC) middleware protecting all API endpoints; MongoDB Atlas cloud database with Mongoose ODM for schema validation; a RESTful Express.js API server with Helmet security headers, CORS policies, and rate limiting; a responsive React.js frontend served via Vite bundler with mobile-adaptive bottom navigation; and deployment on Vercel (frontend) and Render (backend). The platform is accessible at https://hostelease.online.

HostelEase demonstrates a comprehensive understanding of full-stack web development, system design, security best practices, and cloud deployment, representing a significant advancement over existing manual and fragmented hostel management approaches.

Keywords: *MERN Stack, Hostel Management System, React.js, Node.js, MongoDB, JWT Authentication, Role-Based Access Control, RESTful API, Express.js, Room Allocation, Leave Management, Gatekeeper Portal, Audit Log, Vite, Mongoose ODM*

TABLE OF CONTENTS

List of Tabels x

List of Figures xi

List of Abbreviations xii

1 INTRODUCTION 1

1.1 Motivation 1

1.2 Problem Definition 2

1.3 Objectives Of The Project 3

2 LITERATURE SURVEY 4

2.1 Existing System 5

2.2 Disadvantages Of Existing System 6

2.3 Proposed System 7

2.4 Advantages Of The Proposed System 7

3 SYSTEM ANALYSIS 9

3.1 Software Requirement Specification (SRS) 9

3.1.1 User Requirements 9

3.1.2 Functional Requirements 11

3.1.3 Non-Functional Requirements 12

3.1.4 Software Requirements 13

3.1.5 Hardware Requirements 15

3.2 Context Diagram Of The Project 15

4 SYSTEM DESIGN 17

4.1 Introduction 17

4.2 System Design Diagrams 17

4.2.1 System Architecture Diagram 17

4.2.2 Entity-Relationship (ER) Diagram 18

4.2.3 Use Case Diagrams 19

4.2.4 Data Flow Diagram (DFD) Level 0 – HostelEase System 20
4.2.5 Level – 1 DFD HostelEase 21
4.2.6 Leave Request State Diagram 22
4.2.7 Room Allocation and De-Allocation Workflow Diagram 23
4.3 Data Dictionary 24
4.3.1 User Collection 25
4.3.2 Student Collection 26
4.3.3 Room and RoomAllocation Collections 28
4.3.4 Leave Request Collections 29
4.3.5 Supporting Collections 30
5 IMPLEMENTATION & RESULTS 33
5.1 Method Of Implementation 33
5.1.1 React.js (Frontend Implementation) 33
5.1.2 Node.js and Express.js (Backend Implementation) 34
5.1.3 MongoDB and Mongoose (Database) 34
5.1.4 JWT Authentication and RBAC 35
5.2 Implementation Of Modules 35
5.2.1 Authentication Module 35
5.2.2 Student Management Module 37
5.2.4 Leave Management Module 39
5.2.5 GateKeeper Portal Module 40
5.2.6 Issue and Complaint Management Module 41
5.2.7 Notification Module 42
5.2.8 Analytics Dashboard Module 43
5.2.9 Audit Log Module 44
5.2.10 Admin Panel Module 44
5.3 Output Screens 45

5.3.1 Landing Page and Registration 45
5.3.2 Login Page of HostelEase 46
5.3.3 Dashboards 46
5.3.4 Student Room Status 48
5.3.4 Student Apply Leave 48
5.3.5 Student Leave Tracking 49
5.3.6 Student Leave Tracking 49
5.3.7 Student List 50
5.3.8 Room Allocation 50
5.3.9 Leave Approvals 51
5.3.10 Leave Analytics 51
5.3.11 Notifications 52
5.3.12 Issues Management 53
5.3.13 Mobile Application Screens 53
5.4 Results And Analysis 54
6 TESTING & VALIDATION 56
6.1 Introduction To Testing 56
6.2 Features To Be Tested 56
6.2.2 Student and Room Management 56
6.2.3 Leave Management System 56
6.2.4 Complaint and Issue Handling 57
6.2.5 Notification System 57
6.2.6 Admin and Warden Controls 57
6.2.7 Security Testing 57
6.2.8 Performance Testing 57
6.3 Features Not To Be Tested 57
6.4 Testing Tools And Environment 58

6.5 Design Of Test Cases And Scenarios 58

7 CONCLUSION & FUTURE ENHANCEMENT 60

8 REFERENCES 61

LIST OF TABLES

Table 3.1 Functional Requirements of HostelEase..... 11

Table 3.2 Non-Functional Requirements of HostelEase..... 12

Table 3.3 Software Requirement Specifications..... 13

Table 3.4 Hardware Requirement Specifications..... 15

Table 4.1 User Collection Data Dictionary……..... 25

Table 4.2 Student Collection Data Dictionary..... 26

Table 4.3 Room Collection Data Dictionary..... 28

Table 4.4 Room Allocation Collection Data Dictionary..... 28

Table 4.5 Leave Request Collection Data Dictionary..... 29

Table 4.6 Issue Collection Data Dictionary..... 30

Table 4.7 Notification Collection Data Dictionary..... 32

Table 5.1 API Systems and Performance Benchmarks……... 55

Table 6.1 Test Scenarios and Results……..... 58

LIST OF FIGURES

Figure 3.1 Context Diagram of HostelEase 16

Figure 4.1 Three Tier System Architecture Diagram 18

Figure 4.2 Entity Relationship Diagram (ER) ……..... 19

Figure 4.3 Use Case Diagrams ……..... 20

Figure 4.4 DFD level 0 HostelEase System 21

Figure 4.5 DFD level 1 HostelEase System……..... 22

Figure 4.6 State Diagram of Leave Request Processing 23

Figure 4.7 Room Allocation & De-Allocation Workflow……..... 24

Figure 5.1 Hostel Landing Page……..... 46

Figure 5.2 Login Page for HostelEase……..……..... 46

Figure 5.3 Student Dashboard..... 47

Figure 5.4 Warden Dashboard..... 47

Figure 5.5 Gatekeeper Dashboard..... 48

Figure 5.6 Student Room Status..... 48

Figure 5.7 Student Apply Leave..... 49

Figure 5.8 Student Leave Tracking..... 49

Figure 5.9 Add a New Student..... 50

Figure 5.10 Registered Students..... 50

Figure 5.11 Room Allocation..... 51

Figure 5.12 Leave Approvals..... 51

Figure 5.13 Leave Analytics..... 52

Figure 5.14 Warden Portal Notifications..... 52

Figure 5.15 Student Portal Notifications..... 52

Figure 5.16 Issues Management……..... 53

Figure 5.17 Mobile Application View..... 53

LIST OF ABBREVIATIONS

Abbreviation		Full Form
MERN	—	MongoDB, Express.js, React.js, Node.js
API	—	Application Programming Interface
HMS	—	Hostel Management System
REST	—	Representational State Transfer
JWT	—	JSON Web Token
RBAC	—	Role-Based Access Control
ODM	—	Object Document Mapper
CRUD	—	Create, Read, Update, Delete
UI	—	User Interface
UX	—	User Experience
DFD	—	Data Flow Diagram
SPA	—	Single-Page Application
ER	—	Entity-Relationship
CORS	—	Cross-Origin Resource Sharing
HTTP	—	HyperText Transfer Protocol
HTTPS	—	HyperText Transfer Protocol Secure
SHA	—	Secure Hash Algorithm
CSE	—	Computer Science & Engineering
HOD	—	Head of Department
SRS	—	Software Requirements Specification
HTTP	—	Hypertext Transfer Protocol
JSON	—	JavaScript Object Notation

1 INTRODUCTION

The management of residential hostels in educational institutions is a multifaceted administrative challenge that involves handling student registrations, room assignments, leave permissions, maintenance complaints, and gate security simultaneously. Traditionally, these operations have relied on manual record-keeping physical registers, paper-based applications, and face-to-face interactions which are not only time-consuming but also highly susceptible to errors, data loss, and mismanagement.

With the rapid advancement of web technologies and cloud computing, there exists an immense opportunity to digitalize and automate hostel administrative processes. HostelEase is designed to meet this challenge by providing a centralized, web-based Hostel Management System (HMS) that unifies all hostel operations into a single integrated platform, accessible from any device with an internet connection.

HostelEase is built upon the MERN stack, MongoDB for the database, Express.js for the backend API server, React.js for the frontend user interface, and Node.js as the runtime environment. This technology stack enables the development of a robust, scalable, and maintainable full-stack application with a unified JavaScript ecosystem across both client and server. The system is designed to serve four distinct types of users — Administrators, Wardens, Students, and Gatekeepers each accessing a personalized dashboard with role-specific functionalities.

1.1 MOTIVATION

The motivation behind HostelEase stems from first-hand observation of the operational inefficiencies present in hostel management at engineering colleges. Wardens maintain handwritten ledgers to track room occupancies, students submit physical leave forms that require multiple rounds of follow-up, fee receipts are generated manually without digital records, and maintenance complaints are often verbal and therefore untracked and unresolved.

These inefficiencies lead to multiple problems: critical data is frequently lost or misplaced, students experience delays in approvals, wardens spend disproportionate time on administrative tasks instead of welfare activities, and administrators lack real-

time visibility into hostel operations. The absence of a gatekeeper management system means that student exits are not reliably tracked, creating security vulnerabilities.

Furthermore, existing commercial Hostel Management Software is often expensive, complex, or not tailored to the specific requirements of Indian engineering colleges with their unique academic batch structures, multi-department compositions, and integration needs. HostelEase is motivated by the need for an affordable, open, and purpose-built solution that can be rapidly deployed and customized to institutional requirements.

1.2 PROBLEM DEFINITION

The core problem addressed by HostelEase is the fragmented and manual nature of hostel management in academic institutions, which leads to the following specific challenges:

- Absence of a centralized student database with searchable profiles, academic batch information, and medical details.
- Manual room allocation processes prone to errors such as double-allocation of beds or incorrect bed numbering.
- Paper-based leave request workflows that create delays, lack transparency, and cannot be tracked in real time.
- Fee payment records maintained in physical ledgers with no digital audit trail or reporting capability.
- Maintenance complaints submitted verbally or on paper, resulting in untracked resolution times and repeated complaints for the same issues.
- No formal system for gate security personnel to verify and log student exits, creating accountability gaps.
- Absence of analytics and reporting tools that would allow wardens and administrators to make data-driven decisions.
- No notification mechanism to inform students of leave approvals, room allocations, or complaint resolutions in real time.

HostelEase directly addresses each of these challenges through dedicated, integrated modules that automate, digitalize, and secure hostel management workflows.

1.3 OBJECTIVES OF THE PROJECT

The primary objective of this project is to design and develop HostelEase, a centralized, digital hostel management platform that eliminates the inefficiencies of the existing manual system. The specific objectives are:

1. To design and develop a comprehensive, role-based Hostel Management System accessible via a web browser on desktops and mobile devices.
2. To implement a secure, JWT-based authentication system with role-based access control (RBAC) that prevents unauthorized access to administrative and student data.
3. To provide a complete Student Management module enabling wardens to register, search, update, and manage student profiles with rich personal and academic information.
4. To develop a Room Management module that enables wardens to create rooms, define bed capacities, and allocate or de-allocate beds to students with full tracking of occupancy status.
5. To build a Leave Management module that supports the full lifecycle of a leave request — submission, approval or rejection by warden, and physical exit verification by the gatekeeper.
6. To create an Issue and Complaint Management module through which students can raise maintenance or hostel-related complaints with categorization and priority levels, and wardens can resolve them with documented responses.
7. To develop an in-app Notification System that delivers real-time alerts to students and wardens upon occurrence of relevant events such as leave approvals, room allocations, and payment confirmations.
8. To implement an Analytics Dashboard for wardens providing statistical insights into occupancy rates, leave patterns, and complaint resolution metrics.
9. To build a Gatekeeper Portal that enables gate security personnel to search for students by roll number, verify their approved leave status, and formally record student exits with timestamps.
10. To implement a comprehensive Audit Log system that records all administrative actions with user identity, action type, and timestamps for accountability and traceability.

2 LITERATURE SURVEY

A comprehensive review of existing literature and related systems has been conducted to understand current approaches in hostel management, administrative automation, and web-based system design. These studies provide valuable insights into system architecture, security practices, and workflow optimization, which directly influenced the design and implementation of HostelEase.

Sharma and Gupta [1] developed a PHP/MySQL-based hostel management system that enabled student registration and room allocation through a web interface. Their system demonstrated the feasibility of digitizing hostel operations; however, it suffered from scalability limitations, lacked responsive design, and did not support real-time communication features. These limitations are addressed in HostelEase through the adoption of the MERN stack, enabling scalability, responsiveness, and dynamic user interaction.

Singh et al. [2] extended traditional hostel systems by incorporating modules for fee management and complaint tracking. While their system improved operational coverage, it lacked role-based access control, resulting in potential exposure of sensitive administrative data. HostelEase resolves this issue by implementing strict Role-Based Access Control (RBAC), ensuring that each user role (admin, warden, student, gatekeeper) has controlled and secure access.

Commercial systems such as Yugal Hostel Management, eHMS, and InnSync [3] provide comprehensive hostel administration solutions. These platforms offer features like student records, billing, and reporting; however, they are proprietary, involve high licensing costs, and require dedicated infrastructure. Additionally, they often lack modern user interfaces and mobile-first design. HostelEase overcomes these limitations by being a cost-effective, customizable, and web-based solution with a modern UI and cross-platform accessibility.

Reddy and Prasad [4] analyzed digital leave management systems in educational institutions and found that automated workflows significantly reduced processing time compared to manual methods. Their research highlighted the

importance of multi-level approval mechanisms and timely notifications. This directly influenced HostelEase's Leave Management Module, which incorporates structured approval workflows and real-time notification systems.

Kumari et al. [5] demonstrated the effectiveness of the MERN stack in building scalable educational management systems. Their study emphasized the flexibility of MongoDB for handling diverse and dynamic data structures, along with React.js for reusable UI components. HostelEase adopts this architecture to ensure maintainability, scalability, and efficient data handling.

Pal and Bhattacharya [6] examined modern authentication mechanisms and recommended the use of JWT-based authentication combined with bcrypt hashing and rate limiting for enhanced security. HostelEase integrates all these techniques, including secure password hashing, token-based authentication, and request rate limiting to protect against unauthorized access and brute-force attacks.

Ferraiolo et al. [7] introduced the foundational concept of Role-Based Access Control (RBAC), demonstrating its effectiveness in reducing security risks and simplifying permission management in multi-user systems. This principle is central to HostelEase, where middleware-enforced RBAC ensures strict access control across all system modules.

2.1 EXISTING SYSTEM

The existing hostel management approach in most Indian engineering colleges can be characterized as a combination of manual processes and fragmented digital tools:

- Student data is maintained in Excel spreadsheets or physical registers, with no searchability or cross-referencing capabilities.
- Room allocation is tracked manually with the risk of double-booking; no real-time occupancy dashboard exists.
- Leave applications involve physical forms signed by parents, submitted to the warden, and approved manually — a process that takes 1-3 days and provides no real-time status updates to students.

- Fee payments are tracked via physical receipts and separate ledgers, often leading to disputes over payment confirmation.
- Maintenance complaints are reported verbally or via informal WhatsApp messages to wardens, with no tracking, escalation, or resolution confirmation mechanism.
- Gate management is manual, with security personnel relying on physical permission slips that can be easily forged or misplaced.
- No unified notification system exists; students are informed of approvals via phone calls or in-person visits.

2.2 DISADVANTAGES OF EXISTING SYSTEM

The existing manual hostel management approach presents numerous significant disadvantages:

- Data Loss and Corruption: Physical registers are prone to damage, loss, and unauthorized modification, compromising data integrity.
- Inefficiency and Delays: Manual approval processes for leave and room allocation create bottlenecks and reduce warden productivity.
- Lack of Transparency: Students have no real-time visibility into their leave status, room allocation, or payment records.
- Security Vulnerabilities: Absence of a gatekeeper management system means student exits are unverified, creating safety risks.
- No Reporting or Analytics: Administrators and wardens lack data-driven insights into hostel operations, making resource planning difficult.
- Scalability Issues: Manual processes do not scale effectively as hostel occupancy increases or multiple hostels need to be managed.
- No Audit Trail: Actions taken by wardens and administrators are not logged, making accountability difficult to enforce.
- Accessibility Limitations: Physical records can only be accessed on-site, limiting remote management capabilities.

2.3 PROPOSED SYSTEM

To overcome the limitations of existing hostel management systems, the proposed system HostelEase is developed as a comprehensive and unified digital platform. The system digitizes and automates all core hostel administrative processes into a single, integrated solution.

The system is developed using the MERN stack, ensuring scalability, flexibility, and efficient data handling. It is deployed on cloud infrastructure, enabling secure access from any device through a web browser.

The proposed system provides the following features:

- Unified platform integrating all hostel operations in one system
- Role-based access control for admin, warden, student, and gatekeeper
- Real-time notifications for leave requests, approvals, and issue updates
- Automated leave management with multi-level approval workflow
- Digital room allocation and occupancy tracking
- Secure authentication using JWT and encrypted password storage
- Issue/complaint management with status tracking
- Gatekeeper portal for recording student entry and exit
- Analytics dashboard for monitoring hostel operations
- Audit logging for tracking system activities and ensuring accountability
- User-friendly and responsive interface for seamless usage across devices

The system is designed to eliminate manual paperwork, reduce administrative delays, improve transparency, and provide a secure and efficient environment for hostel management.

2.4 ADVANTAGES OF THE PROPOSED SYSTEM

HostelEase proposed design offers the following key advantages over the existing manual approach and all reviewed existing systems:

- Centralized Data Management: All hostel data — student profiles, room allocations, leave records, payments, and complaints — is stored in a single, searchable, cloud-hosted database.
- Real-Time Operations: Wardens can approve or reject leave requests instantly, and students receive immediate notifications, reducing the entire process to minutes.

- Enhanced Security: JWT authentication, bcrypt password hashing, rate limiting, Helmet security headers, and CORS policies provide multi-layered protection against unauthorized access and common web attacks.
- Role-Based Access: Each user type sees only the functionality relevant to their role, preventing unauthorized access to sensitive data.
- Formal Gate Management: The Gatekeeper Portal enables structured exit verification and logging, closing the security gap in student movement tracking.
- Data-Driven Decision Making: Analytics dashboards provide wardens with occupancy trends, payment summaries, and complaint resolution metrics.
- Accountability and Traceability: Comprehensive audit logs record every administrative action, enabling post-hoc review and accountability enforcement.
- Scalability: The cloud-based architecture can scale to support multiple hostels, thousands of students, and growing data volumes without infrastructure changes.
- Mobile Accessibility: The responsive React.js frontend works seamlessly on smartphones and tablets, enabling access from anywhere.
- Cost-Effective: Being open-source and self-hosted on free-tier cloud infrastructure, HostelEase eliminates licensing costs entirely.

3 SYSTEM ANALYSIS

System analysis is the process of studying the current system or the problem domain to identify the requirements, constraints, and objectives of the proposed software system. This chapter presents the Software Requirements Specification (SRS) for HostelEase, covering user requirements, functional requirements, non-functional requirements, software requirements, and hardware requirements.

3.1 SOFTWARE REQUIREMENT SPECIFICATION (SRS)

The Software Requirement Specification (SRS) defines the functional and non-functional requirements of the system. It serves as a reference for developers, designers, and stakeholders throughout the development lifecycle.

The SRS for HostelEase includes user requirements, software requirements, and hardware requirements necessary for the proper functioning of the system.

3.1.1 User Requirements

The HostelEase system serves four distinct categories of users, each with specific requirements:

Administator Requirements

- The administrator shall be able to log into the system using a secure email and password combination.
- The administrator shall be able to create new warden accounts with appropriate credentials.
- The administrator shall have visibility into all users registered in the system with the ability to deactivate accounts.
- The administrator shall be able to view system-wide audit logs and operational statistics.

Warden Requirements

- The warden shall be able to register new students into the hostel system with complete personal, academic, and medical details.
- The warden shall be able to create and manage room records with room numbers and bed capacities.
- The warden shall be able to allocate and de-allocate rooms and specific beds to students with automatic occupancy tracking.

- The warden shall be able to view, approve, or reject student leave requests with the option to add remarks.
- The warden shall be able to record student fee payments with details including amount, payment mode, academic year, and receipt number.
- The warden shall be able to view and resolve maintenance issues raised by students with documented resolution responses.
- The warden shall be able to access an analytics dashboard showing occupancy rates, leave statistics, and payment summaries.
- The warden shall be able to view a complete audit log of all actions performed in the system.

Student Requirements

- The student shall be able to log into the system using credentials provided by the warden.
- The student shall be able to view their personal profile, room allocation details, and academic information.
- The student shall be able to apply for leave by specifying leave type, date range, destination, reason, and emergency contact.
- The student shall be able to track the status of their leave applications in real time.
- The student shall be able to view their fee payment history and pending dues.
- The student shall be able to raise maintenance or hostel-related complaints with category, priority, and description.
- The student shall receive in-app notifications for leave approvals, room allocations, and payment confirmations.
- The student shall be able to reset their password via a secure email-based token link.

GateKeeper Requirements

- The gatekeeper shall be able to log into a dedicated portal using gatekeeper credentials.
- The gatekeeper shall be able to search for a student by roll number to view their current leave status.
- The gatekeeper shall be able to formally record a student's exit when they have an approved leave, capturing the exit timestamp.

- The gatekeeper shall be able to deny a student exit with a documented reason.

3.1.2 Functional Requirements

Functional requirements define the specific behaviours and capabilities that the HostelEase system must exhibit.

Table 3.1 Functional Requirements of HostelEase

ID	Requirement	Description
FR-01	User Authentication	The system shall authenticate users via email and password with JWT token generation upon successful login.
FR-02	Role-Based Access	The system shall enforce role-based access control, restricting API endpoints based on user role (admin, warden, student, gatekeeper).
FR-03	Student Registration	The warden shall be able to register new students with full personal and academic details.
FR-04	Student Profile Management	Wardens shall be able to search, view, edit, and delete student records.
FR-05	Room Management	The system shall support creating rooms with room numbers and total bed capacities.
FR-06	Room Allocation	The system shall allocate specific beds to students and track occupancy dynamically.
FR-07	Leave Application	Students shall be able to submit leave requests with type, dates, destination, reason, and emergency contact.
FR-08	Leave Approval	Wardens shall be able to approve or reject leave requests with remarks.

FR-09	Gatekeeper Exit Tracking	Gatekeepers shall record approved student exits with timestamp and denial reason if applicable.
FR-10	Payment Recording	Wardens shall record student fee payments with amount, mode, academic year, and receipt.
FR-12	Issue Resolution	Wardens shall resolve reported issues with a documented resolution response.
FR-13	Notifications	The system shall generate and deliver in-app notifications for key events.
FR-14	Analytics Dashboard	The system shall provide statistical dashboards for wardens with occupancy, leave, and payment data.
FR-15	Audit Logging	The system shall automatically log all administrative actions.
FR-16	Password Reset	Students shall be able to reset their password via a tokenized email link valid for 15 minutes.

3.1.3 Non-Functional Requirements

Non-Functional requirements define the specific behaviours and capabilities that the HostelEase system must exhibit.

Table 3.2 Non-Functional Requirements

Quality Attribute	Requirement Description
Security	All API endpoints shall be protected by JWT authentication. Passwords shall be hashed using bcrypt with a minimum of 10 salt rounds. Rate limiting shall prevent brute-force attacks.

Performance	API response time for standard queries shall not exceed 500ms under normal load. The React.js frontend shall achieve a Lighthouse performance score above 80.
Scalability	The system architecture shall support horizontal scaling of the Node.js backend. MongoDB Atlas shall be used to support auto-scaling of the database cluster.
Availability	The system shall target 99.5% uptime via Render (backend) and Vercel (frontend) cloud platforms.
Maintainability	Code shall follow modular MVC architecture with separate folders for models, controllers, and routes. Descriptive variable and function names shall be used throughout.
Usability	The UI shall be mobile-responsive and accessible on screens from 320px width. Navigation shall require no more than 3 clicks to reach any primary feature.
Reliability	All database write operations shall use Mongoose validation to prevent corrupt data storage. Error handlers shall catch all unhandled exceptions and return meaningful HTTP status codes.
Portability	The frontend shall be a cross-browser compatible SPA running on Chrome, Firefox, Edge, and Safari. The backend shall run on any Node.js-compatible server environment.

3.1.4 Software Requirements

The complete software requirements for the development and deployment of HostelEase:

Table 3.3 Software Requirements Specification

Component	Technology/Tool	Version/Details
Frontend Runtime	React.js with Vite	React 18.x, Vite 5.x

Styling Framework	Tailwind CSS	v3.x
HTTP Client	Axios	v1.x
Routing	React Router DOM	v6.x
State Management	React Context API	Built-in React
Backend Runtime	Node.js	v20.x LTS
Web Framework	Express.js	v4.x
Database ODM	Mongoose	v8.x
Database	MongoDB	v7.x (Atlas Cloud)
Authentication	JSON web token	v9.x
Password Hashing	Bcrypt.js	v2.x
Email Service	Resend	v6.x
Security Middleware	Helmet.js	v7.x
CORS Middleware	cors	v2.x
Rate Limiting	express-rate-limit	v7.x
Input Sanitization	express-mongo-sanitize	v2.x
Environment Variables	dotenv	v16.x
Version Control	Git + GitHub	Latest
API Testing	Postman	v10.x
Code Editor	Visual Studio Code	Latest
Package Manager	npm	v10.x
Frontend Deployment	Vercel	Latest

Backend Deployment	Render	Latest
Database Cloud	MongoDB Atlas	Free / Shared Tier
Operating System (Dev)	Windows 11 / macOS	64-bit

3.1.5 Hardware Requirements

The following table details the hardware requirements for deploying and accessing HostelEase:

Table 3.4: Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 (2.0 GHz)	Intel Core i5/i7 (2.5 GHz+)
RAM	4 GB	8 GB or more
Storage	256 GB HDD	512 GB SSD
Display	1280×720 (HD)	1920×1080 (Full HD)
Network	Broadband (10 Mbps)	High-Speed (50 Mbps+)
Browser	Chrome 90+ / Firefox 88+	Chrome / Edge (latest)
Mobile (End-user)	Android 8.0+ / iOS 13+	Android 11+ / iOS 15+
Server (Cloud)	Render Free Tier (0.5 CPU, 512 MB RAM)	Render Starter Tier (1 CPU, 512 MB RAM)

3.2 CONTEXT DIAGRAM OF THE PROJECT

The context diagram (Fig. 3.1) represents HostelEase as a single processing system node that interacts with four external entities: Student, Warden, Administrator, and Gatekeeper. All information flows between these external entities and the HostelEase system pass through well-defined interfaces.

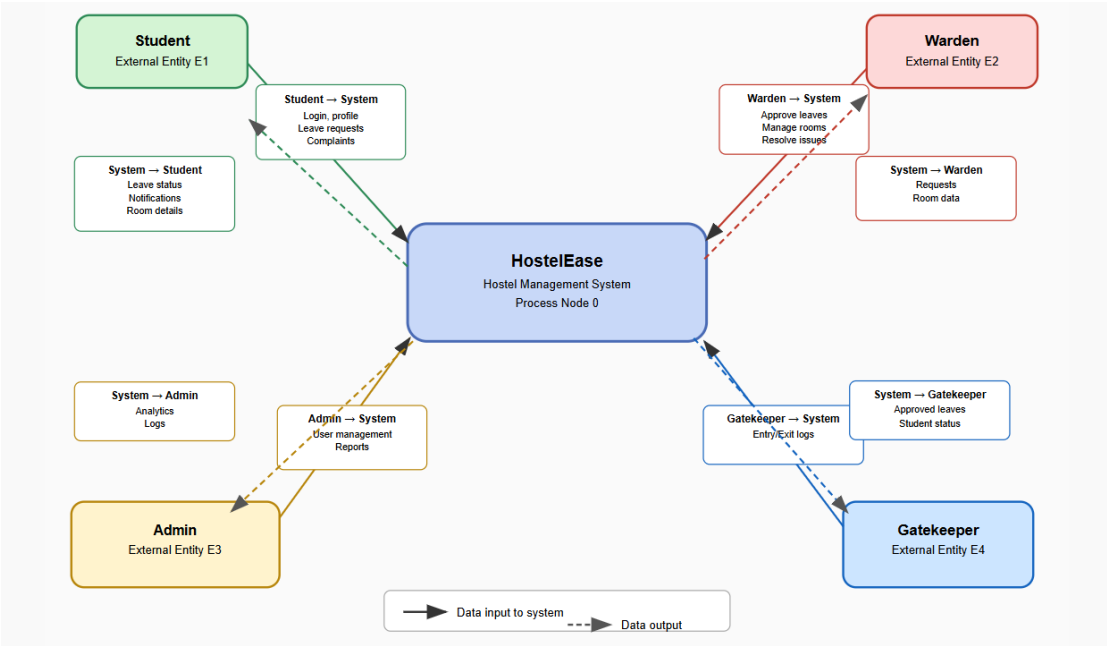

Fig. 3.1: Context Diagram of HostelEase

Student↔HostelEase: Inputs – registration/login data, profile updates, leave requests, complaint submissions. Outputs – leave approval/rejection status, room allocation details, complaint status updates, notifications and announcements.

Warden↔HostelEase: Inputs – leave approvals/rejections, room allocation updates, complaint resolution actions. Outputs – student leave requests, room occupancy details, complaint reports, notification alerts.

Administrator↔HostelEase: Inputs – user account management, room configuration data, system settings, announcement creation. Outputs – analytics dashboards, system reports, user activity logs, approval summaries.

Gatekeeper↔HostelEase: Inputs – student entry/exit records, verification actions for leave permissions. Outputs – approved leave details, student status verification, real-time alerts and notifications.

4 SYSTEM DESIGN

4.1 INTRODUCTION

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy specified requirements. For HostelEase, the design phase translates the functional and non-functional requirements identified in the previous chapter into concrete architectural decisions, database schemas, API structures, and user interface layouts. The design follows the Model-View-Controller (MVC) architectural pattern, separating concerns between data models, business logic controllers, and view presentation.

The HostelEase architecture is structured as a client-server application where the React.js frontend (client) communicates with the Express.js backend (server) via a RESTful HTTP API. The backend persists data in MongoDB Atlas and enforces authentication and authorization at the middleware level before passing requests to controllers. This separation of concerns enables independent development, testing, and scaling of frontend and backend components.

4.2 SYSTEM DESIGN DIAGRAMS

The system diagrams provide a visual representation of the structure, data flow, and operational processes of the HostelEase platform. These diagrams help in understanding how different components of the system interact with each other and how data moves across various modules.

The diagrams included in this section illustrate the system architecture, database relationships, booking lifecycle, and core workflows such as payment processing and dispute resolution. Together, they provide a comprehensive view of the system design and its functional behavior.

4.2.1 System Architecture Diagram

The HostelEase system follows a three-tier architecture for efficient and scalable operation:

Presentation Tier:

The frontend is built using React.js with Vite as a Single-Page Application (SPA). It handles routing through React Router, manages authentication using Context API, and communicates with the backend via Axios. The UI dynamically adapts based

on user roles (Admin, Warden, Student, Gatekeeper) and is hosted on Vercel for fast and reliable access.

Application Tier:

The backend is developed using Node.js and Express.js, providing RESTful APIs for modules such as authentication, student management, leave, rooms, analytics, notifications, audit, issues, and gate management. It ensures security through JWT-based authentication, role-based access control, and middleware like Helmet, rate limiting, and CORS. The server is deployed on Render.

Data Tier:

MongoDB Atlas is used as the cloud database, storing data in collections such as User, Student, Room, RoomAllocation, LeaveRequest, Issue, Notification, AuditLog, and HostelRegistration. Mongoose ODM is used for schema validation, data consistency, and efficient querying with indexing.

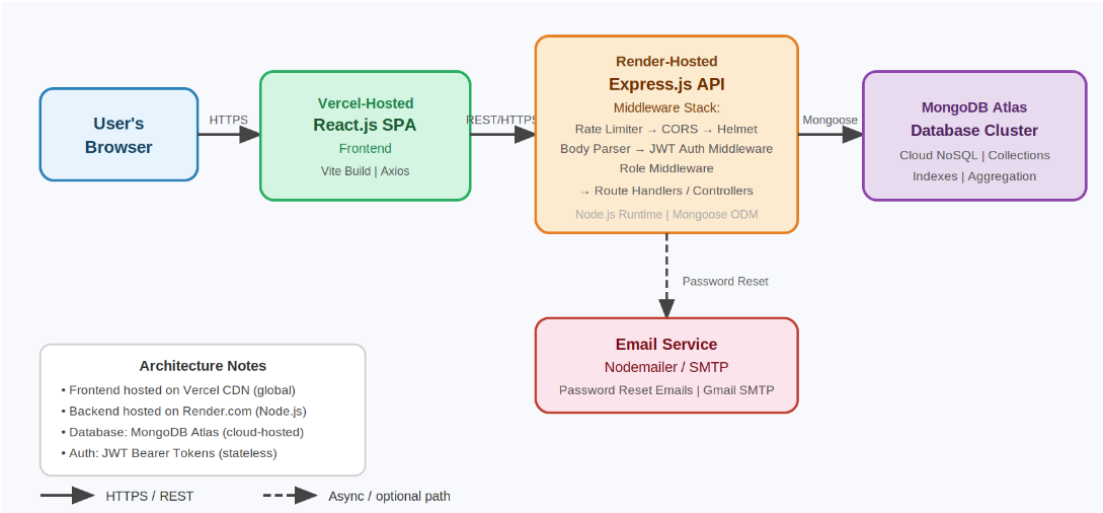

Fig. 4.1: Three-Tier System Architecture Diagram

4.2.2 Entity-Relationship (ER) Diagram

The ER diagram of HostelEase shows the main entities and their relationships. The User entity acts as the central authentication unit and is linked to Student, RoomAllocation, LeaveRequest, Notification, and AuditLog. The Student entity stores additional details and is associated with RoomAllocation, LeaveRequest, and Issue. The Room entity has a one-to-many relationship with RoomAllocation. A student can have multiple leave requests and issues, while each user can receive multiple

notifications. The AuditLog records administrative actions. These relationships ensure structured and efficient data management in the system.

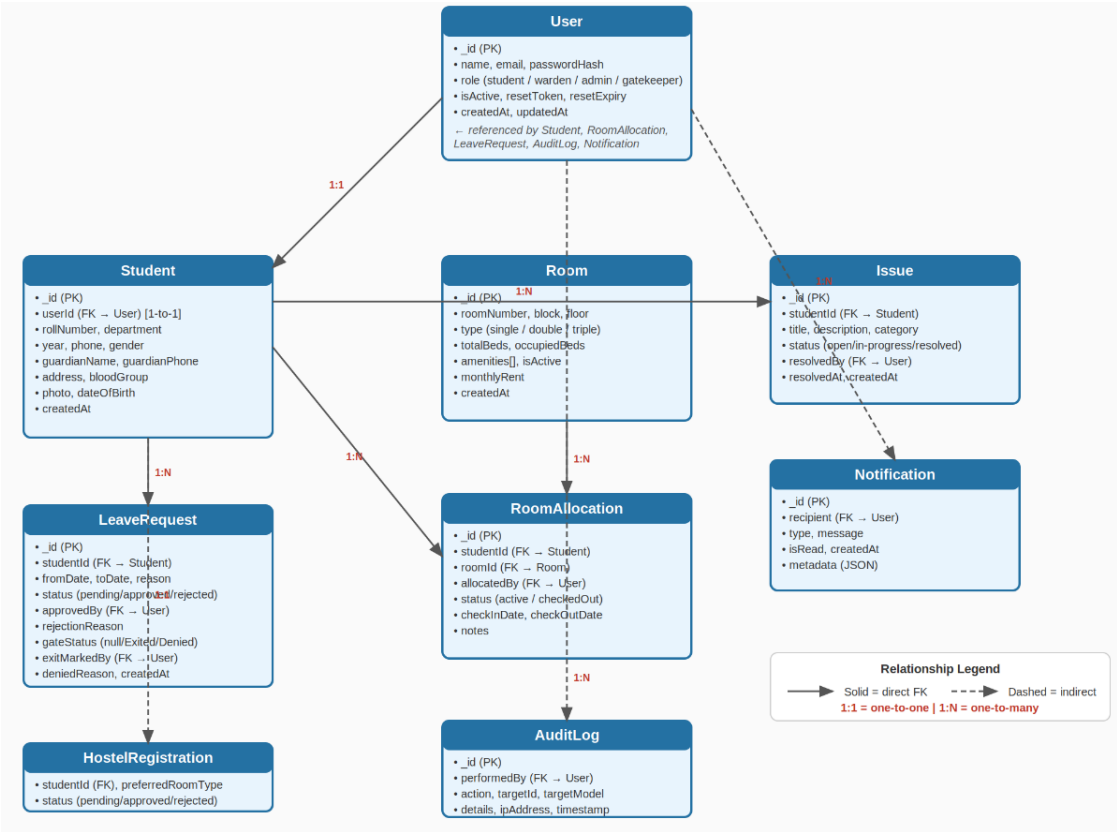

Fig. 4.2: Entity Relationship (ER) Diagram

4.2.3 Use Case Diagrams

The use case diagram represents the overall functionality of the HostelEase system by illustrating the interactions between multiple actors and the system. The system consists of four primary actors: Student, Warden, Gatekeeper, and Admin, each performing specific operations based on their roles.

The Student interacts with the system to perform activities such as logging in, viewing the dashboard, checking room status, applying for leave, viewing leave status, raising issues, and accessing notifications. These functionalities allow students to manage their hostel-related activities efficiently.

The Warden is responsible for managing hostel operations. The warden can add, edit, or delete student records, create and manage rooms, allocate or deallocate rooms, approve or reject leave requests, resolve issues raised by students, monitor analytics, view audit logs, and send notifications. This role ensures smooth administrative control within the hostel.

The Gatekeeper interacts with the system through a dedicated portal. The gatekeeper can log in, search students using roll numbers, verify leave status, and mark student exits (either approved or denied with a reason). This ensures controlled and secure entry and exit management.

The Admin oversees the entire system and has higher-level control. The admin can create warden accounts, view all users, manage user access, and deactivate accounts when necessary. This role maintains system integrity and user management.

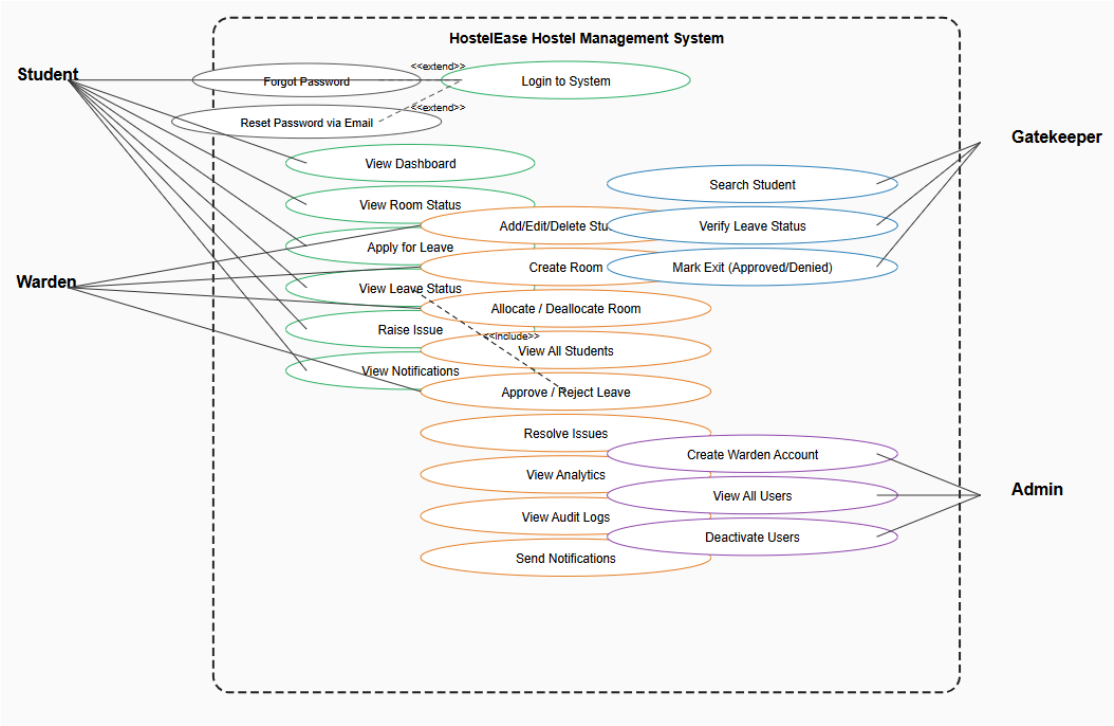

Fig. 4.3: Use Case Diagram

4.2.4 Data Flow Diagram (DFD) Level 0 – HostelEase System

The Level 0 Data Flow Diagram represents the overall structure of the HostelEase system as a single process. It shows how external entities such as Student, Warden, Administrator, and Gatekeeper interact with the system. Students submit leave requests, complaints, and login details, while the system provides notifications and approval updates in response.

The Warden handles approvals and room allocation, the Administrator manages users and system activities, and the Gatekeeper verifies student exits. All the data is stored and managed in a central database (MongoDB Atlas). The diagram clearly shows the flow of data into the system using solid arrows and output from the system using dashed arrows.

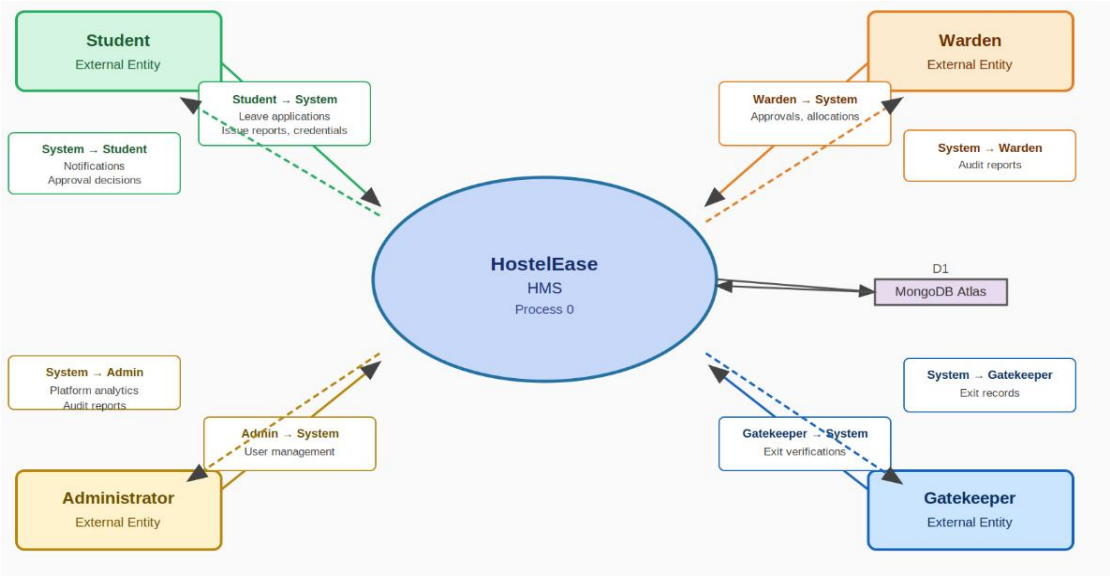

Fig. 4.4: DFD Level 0 – HostelEase System

4.2.5 Level – 1 DFD HostelEase

The Level 1 Data Flow Diagram provides a detailed breakdown of the HostelEase system by dividing it into multiple sub-processes. These include Authentication & Authorization, Student Management, Room Management, Leave Management, Issue Management, Notification Management, and Analytics & Audit. Each process handles a specific function of the system and interacts with external entities such as Student, Warden, Admin, and Gatekeeper.

All processes communicate with a central database (MongoDB Atlas) to store and retrieve data. The system manages user authentication, student records, room allocation, leave requests, and issue tracking efficiently. The Notification Management process plays a key role by generating real-time alerts based on system events, while the Analytics & Audit process provides reports and insights. The diagram clearly shows how data flows between processes, external entities, and the database, ensuring smooth and organized hostel management.

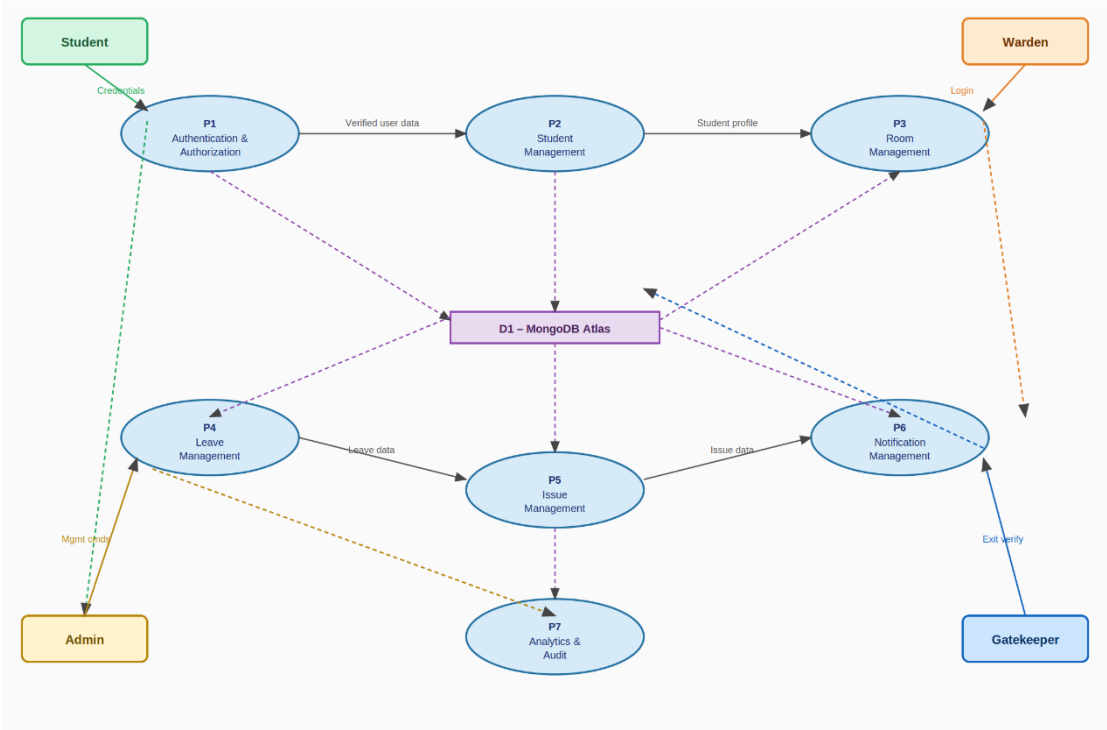

Fig. 4.5: Level-1 DFD – Hostel Ease

4.2.6 Leave Request State Diagram

The Leave Request State Diagram represents the different states through which a student's leave request passes in the HostelEase system. The process begins with the Draft state, where the student creates a leave request. Once submitted, the request moves to the Pending state, where it awaits approval from the warden.

From the Pending state, the request can either be Approved or Rejected based on the warden's decision. If approved, the gatekeeper verifies the student's exit, leading to either Exited or Exit Denied states. If rejected, the system stores the rejection reason and notifies the student. Notifications are generated at each important stage, and the process ends when the request reaches a final state such as or Rejected.

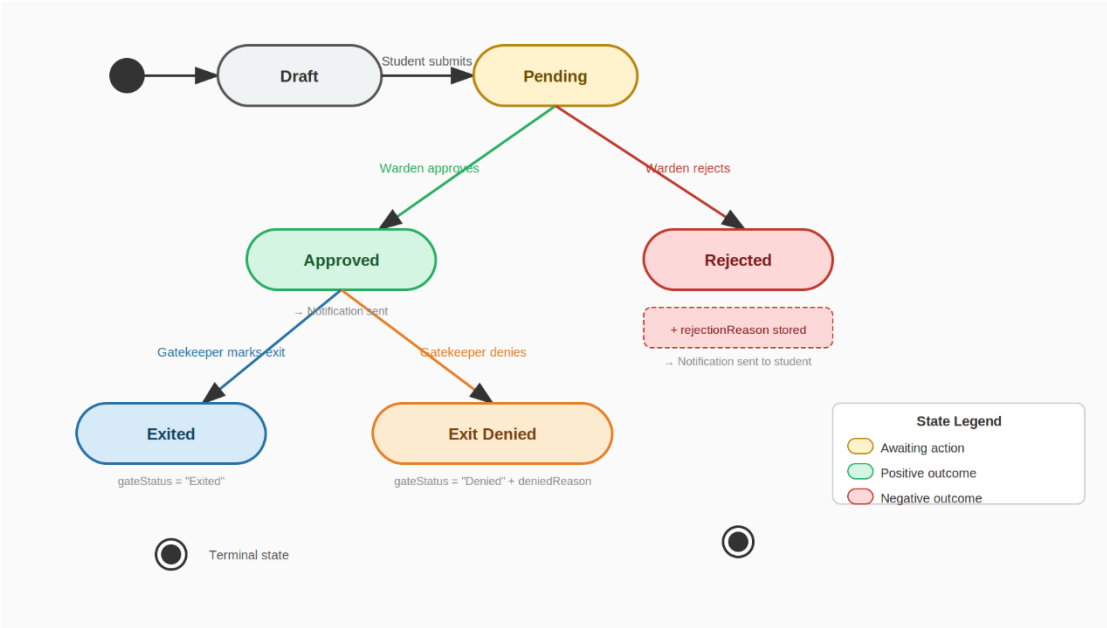

Fig. 4.6: State Diagram for Leave Request Processing

4.2.7 Room Allocation and De-Allocation Workflow Diagram

The Room Allocation Workflow diagram illustrates the step-by-step process of assigning rooms to students in the HostelEase system. The process begins when the warden initiates room allocation. The system first checks whether the student already has an active room allocation; if yes, an error is returned. If not, the system then verifies whether the selected room has available beds. If the room is full, the allocation is denied with an appropriate error message.

If both conditions are satisfied, the system proceeds to create a room allocation record and updates the number of occupied beds using atomic operations to maintain data consistency. A notification is then sent to the student confirming the room allocation. The diagram also shows the de-allocation process, where the student checks out, the allocation status is updated, occupied beds are decremented, and a de-allocation notification is generated. This workflow ensures accurate and efficient room management within the system.

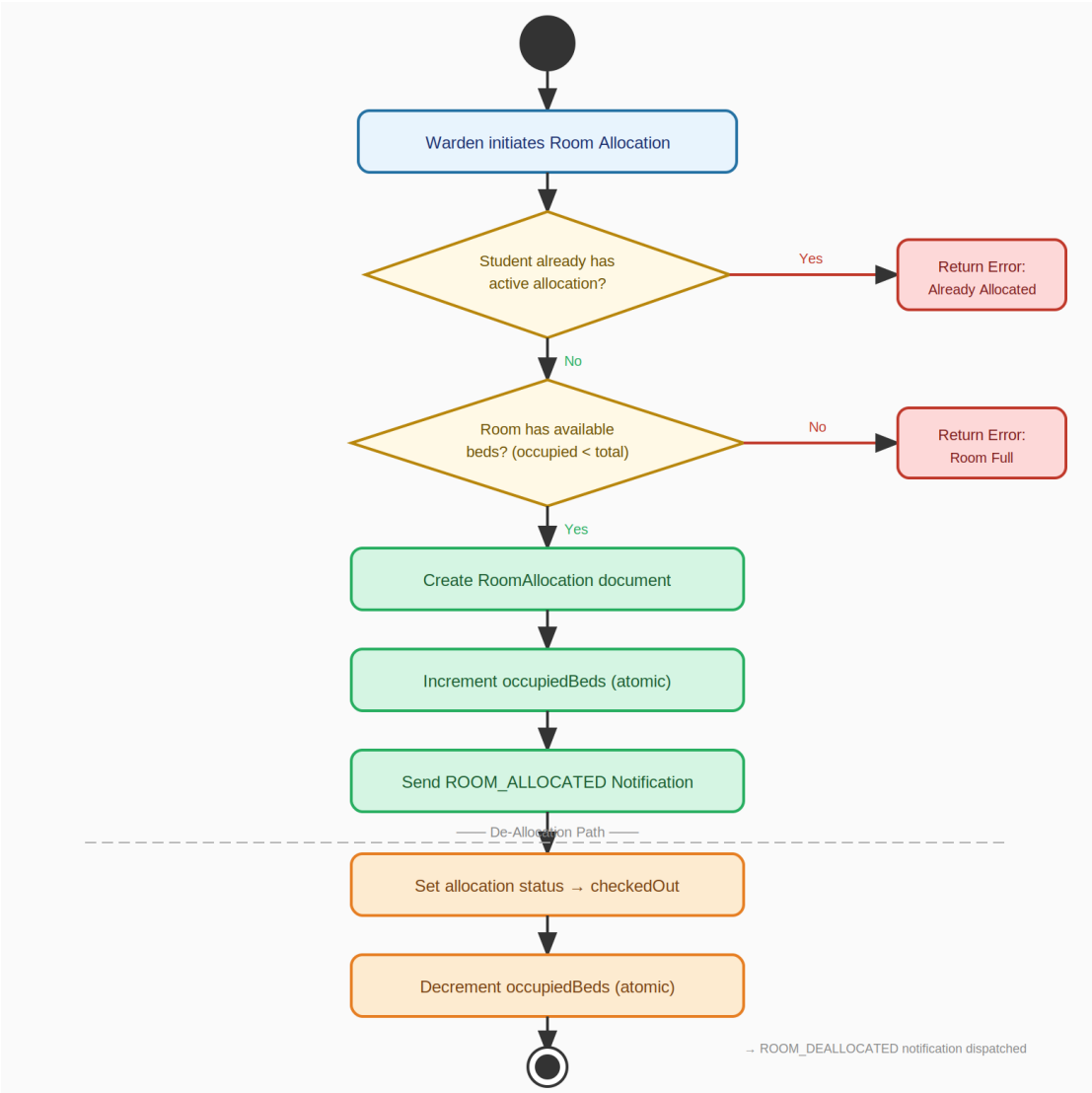

Fig. 4.7: Room Allocation and De-Allocation Workflow

4.3 DATA DICTIONARY

The data dictionary provides a structured description of the database used in the HostelEase Management. It defines the collections, their fields, data types, and purpose. Since the system uses MongoDB, each entity is represented as a collection, and relationships are maintained using references and embedded documents.

This section helps in understanding how data is organized, stored, and accessed within the system.

4.3.1 User Collection

The User collection serves as the authentication foundation for all system users. Every system actor as Admin, Warden, Student, and Gatekeeper has a corresponding User document.

Table 4.1: User Collection – Data Dictionary

Field	Type	Constraints	Description
_id	ObjectId	Auto-generated	Unique MongoDB document identifier.
name	String	Required	Full name of the user.
email	String	Required, Unique	Email address used for authentication.
password	String	Required	Bcrypt-hashed password (12 salt rounds).
role	String	Enum: admin|warden|student|gatekeeper	Determines system access level and available functionality.
resetPassword Token	String	Optional	SHA-256 hashed reset token stored during password reset flow.
resetPassword Expires	Date	Optional	Expiry timestamp for password reset token (15-minute window).
isActive	Boolean	Default: true	Account activation status; deactivated accounts cannot login.
createdAt	Date	Auto	Document creation timestamp

updatedAt	Date	Auto	Last modification timestamp managed by Mongoose timestamps.

4.3.2 Student Collection

The Student collection stores comprehensive personal, academic, and medical information for each registered student, extending the basic User authentication record.

Table 4.2: Student Collection Data Dictionary

Field	Type	Constraints	Description
_id	ObjectId	Auto-generated	Unique identifier.
user	ObjectId (ref: User)	Required, Unique	Reference to the corresponding User document.
avatar	String	Default: "	Base64 data URL or external URL for the student's profile photo.
fullName	String	Required, Trimmed	Student's complete legal name.
rollNumber	String	Required, Unique, Uppercase	College roll number, auto-converted to uppercase.
course	String	Optional	Degree programme (e.g., B.Tech).
department	String	Optional, Indexed	Academic department (e.g., CSE, ECE).

collegeName	String	Optional	Name of the enrolled college or institution.
batch	String	Required, Indexed	Academic batch year (e.g., 2022-2026).
gender	String	Enum: Male|Female|Other	Student's gender.
dateOfBirth	Date	Optional	Student's date of birth.
bloodGroup	String	Optional	Blood group for medical purposes.
phoneNumber	String	Unique (sparse)	Student's personal mobile number.
aadhaarNumber	String	Sparse	Government-issued Aadhaar id
fatherName	String	Optional	Father's full name.
motherName	String	Optional	Mother's full name.
parentContact	String	Optional	Primary parent/guardian contact number.
medicalIssues	String	Default: 'None'	Known medical conditions or allergies.
address	String	Optional, Trimmed	Permanent residential address.

4.3.3 Room and RoomAllocation Collections

The Room collection stores physical room definitions, while RoomAllocation tracks the assignment of students to specific beds within those rooms.

Table 4.3: Room Collection Data Dictionary

Field	Type	Constraints	Description
roomNumber	Number	Required, Unique	Unique integer identifier for the room.
totalBeds	Number	Required	Total number of beds in the room.
occupiedBeds	Number	Default: 0	Count of currently occupied beds. Incremented on allocation, decremented on checkout.

Table 4.4: Room Allocation Collection-Data Dictionary

Field	Type	Constraints	Description
student	ObjectId (ref: Student)	Required	Student who is allocated the bed.
room	ObjectId (ref: Room)	Required	Room in which the bed is located.
bedNumber	String	Required	Specific bed identifier (e.g., A1, B2).
allocatedBy	ObjectId (ref: User)	Optional	Warden who performed the allocation.
checkInDate	Date	Default: now	Date of bed allocation.
checkOutDate	Date	Optional	Date when student checked out.
status	String	active|out	Current allocation status. Unique

4.3.4 Leave Request Collections

The LeaveRequest collection stores the complete lifecycle of a student's leave application, from initial submission through warden approval to gatekeeper exit verification.

Table 4.5: LeaveRequest Collection – Data Dictionary

Field	Type	Constraints	Description
student	ObjectId (ref: Student)	Required, Indexed	Originating student's reference.
leaveType	String	Enum: Casual|Home|Medical|Emergency|Event|Other	Category of leave being requested.
reason	String	Required, Min 10 chars	Detailed reason for leave application.
fromDate	Date	Required	Start date of the leave period.
toDate	Date	Required	End date of the leave period.
totalDays	Number	Computed	Auto-calculated number of leave days.
destination	String	Required	Address or location the student intends to visit.
emergency Contact	String	Required	Contact number reachable during leave.
status	String	Enum: Pending|Approved|Rejected; Indexed	Current approval status of the request.

remarks	String	Optional	Warden's remarks upon approval.
rejectionReason	String	Conditional: required when status=Rejected	Mandatory reason provided when rejecting a request.
approvedBy	ObjectId (ref: User)	Optional	Warden who approved or rejected the request.
exitedAt	Date	Default: null	Timestamp when gatekeeper recorded the student's exit.
exitMarkedBy	ObjectId (ref: User)	Default: null	Gatekeeper who recorded the exit.
gateStatus	String	Enum: Pending|Exited|Denied|null	Physical exit status at the gate.
gateDeniedReason	String	Default: null	Reason provided by gatekeeper when denying exit.

4.3.5 Supporting Collections

Issues

The Issue collection tracks student-reported complaints:

Table 4.6: Issue Collection – Data Dictionary

Field	Type	Constraints	Description
student	ObjectId (ref: Student)	Required, Indexed	Student who raised the complaint.

title	String	Required, Max 120 chars	Brief title describing the issue.
description	String	Required, Min 10 chars	Detailed description of the problem.
category	String	Enum: Maintenance|Cleanliness|Electrical|Plumbing|Security|Food|Internet|Noise|Other	Issue classification for routing and reporting.
priority	String	Enum: Low|Medium|High|Urgent; Default: Medium	Urgency level of the issue.
status	String	Enum: Open|In Progress|Resolved|Closed; Indexed	Current resolution status.
resolution	String	Default: "	Warden's documented resolution response.
resolvedBy	ObjectId (ref: User)	Optional	Warden who resolved the issue.
resolvedAt	Date	Optional	Timestamp of issue resolution.
attachments	[String]	Optional	Array of base64 image strings or URLs for photo evidence.

Notifications

The Notification collection stores in-app alerts delivered to users, with automatic TTL-based deletion after 90 days:

Table 4.7: Notification Collection – Data Dictionary

Field	Type	Constraints	Description
recipient	ObjectId (ref: User)	Required, Indexed	Target user for this notification.
type	String	Enum: {data}	Classification of the notification event.
title	String	Required	Short notification heading.
message	String	Required	Detailed notification content.
isRead	Boolean	Default: false, Indexed	Whether the user has read this notification.
relatedModel	String	Optional	Name of the related Mongoose model (e.g., LeaveRequest).
relatedId	ObjectId	Optional	ID of the related document for deep-linking.
createdAt	Date	TTL: 90 days	Auto-expires after 90 days via MongoDB TTL index.

5 IMPLEMENTATION & RESULTS

5.1 METHOD OF IMPLEMENTATION

HostelEase is developed using the Agile Software Development methodology, where the project is divided into iterative sprints aligned with individual modules. Each sprint follows a structured cycle of requirement analysis, implementation, unit testing, and integration validation. This approach ensures continuous feedback, quick issue resolution, and flexibility to make improvements throughout the development process.

The system follows the MVC (Model-View-Controller) architecture to maintain a clear separation of concerns. Mongoose schemas are used for defining models and handling database interactions, controllers manage the business logic, and routes connect HTTP requests to the appropriate controllers through middleware. The React.js frontend serves as the View layer, providing user-friendly, role-based interfaces and interacting with the backend via APIs.

The application is deployed using Vercel for the frontend and Render for the backend, ensuring high availability and scalability.

5.1.1 React.js (Frontend Implementation)

React.js is a declarative, component-based library used in HostelEase to build responsive user interfaces. The project uses React 18 for improved rendering performance and is developed with Vite, which provides fast Hot Module Replacement (HMR) for efficient development. The frontend is structured into feature-based directories, including pages for different user roles, reusable components, and a global authentication context.

The frontend is organized into feature-based directories: /src/pages contains route-level components organized by user role (admin, warden, student, gatekeeper, auth, home); /src/components contains shared reusable components such as role-specific navigation bars, the notification bell, protected route wrapper, and the mobile top bar; and /src/context contains the global authentication context provider.

React Router DOM v6 handles client-side routing with the ProtectedRoute component acting as a guard that checks the user's authentication token and role before

rendering the requested page, redirecting to login if credentials are missing or insufficient. The Axios library is configured with a base URL and automatic.

5.1.2 Node.js and Express.js (Backend Implementation)

Node.js is an asynchronous, event-driven runtime built on Chrome's V8 engine, making it ideal for scalable network applications. Its non-blocking I/O model allows HostelEase to efficiently handle multiple concurrent HTTP requests, ensuring smooth API performance even under high load. Express.js is used as the backend framework, providing a lightweight and flexible structure for building RESTful APIs.

The Express application is secured and optimized through a structured middleware pipeline, including helmet for HTTP security headers, mongoSanitize to prevent NoSQL injection, cors for controlled access, and rate limiters to mitigate DDoS and brute-force attacks. It also uses express.json for parsing request bodies, along with JWT-based authentication and role-based authorization for protected routes. The server connects to MongoDB Atlas via Mongoose with configured timeouts for reliability, and includes a SIGTERM handler to ensure graceful shutdown and prevent data corruption during deployments.

5.1.3 MongoDB and Mongoose (Database)

MongoDB is a NoSQL document database that stores data in flexible, JSON-like BSON documents, allowing HostelEase to handle diverse student records without rigid schema constraints. Its schema-less design makes it easy to adapt to changing data requirements without complex migrations. Mongoose is used as the Object Document Mapper (ODM), enabling schema definitions, validation, type casting, and query handling. In HostelEase, strict schemas are defined with built-in validations and timestamps to automatically manage createdAt and updatedAt fields, ensuring data consistency and integrity.

MongoDB Atlas serves as the cloud database platform, offering features like automatic backups, monitoring, and scalability through sharding. The M0 free tier is suitable for small-scale deployments, while higher tiers support larger institutions. HostelEase also implements query optimization using indexes, such as filtering by department and batch in the Student collection, compound indexes in LeaveRequest

and Issue collections, and a TTL index in the Notification collection to automatically delete records after 90 days, improving performance and storage efficiency.

5.1.4 JWT Authentication and RBAC

JWT is used in HostelEase for secure, stateless authentication, eliminating the need for server-side sessions. After login, a token containing the user ID and role is generated with an expiry and stored on the client side. This token is included in the Authorization header for all API requests. The verification middleware checks the token's validity, decodes it, and enforces role-based access to protect restricted routes.

Passwords are secured using bcryptjs with strong hashing to prevent unauthorized access. For password reset, a cryptographically secure random token is generated, hashed, and stored in the database with a short expiry time. The original token is sent to the user, and during reset, it is rehashed and verified, ensuring a safe and reliable password recovery process.

5.2 IMPLEMENTATION OF MODULES

The module implementation in HostelEase is designed in a modular way, where each feature is developed independently following the MVC pattern. Each module includes proper validation, API integration, and role-based access control to ensure secure and efficient system functionality.

5.2.1 Authentication Module

The Authentication Module in HostelEase handles login, session management, and password recovery for all user roles. During login, user credentials are validated using bcrypt, and a JWT token is generated for secure access. Invalid or inactive users are restricted with proper HTTP responses to ensure security.

The password reset process uses a secure token generated with Node.js crypto, hashed and stored with a short expiry. The reset link is sent via email, and the system verifies the token before allowing password updates. Route guards and RBAC further ensure that only authorized users can access specific pages and functionalities.

Code Implementation: authController.js

```
// controllers/authController.js
const crypto = require("crypto");
```

```
const bcrypt = require("bcryptjs");
const jwt    = require("jsonwebtoken");
const User   = require("../models/User");

// LOGIN
exports.login = async (req, res, next) => {
try {
const { email, password } = req.body;
const user = await User.findOne({ email }).select("+password");
if (!user) return res.status(401).json({ message: "Invalid credentials" });
if (!user.isActive) return res.status(403).json({ message: "Account
deactivated" });
const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) return res.status(401).json({ message: "Invalid credentials"
});
const token = jwt.sign({ id: user._id, role: user.role },
process.env.JWT_SECRET, { expiresIn: "7d" });
res.json({ token, role: user.role, name: user.name });
} catch (err) { next(err); }
};

// FORGOT PASSWORD
exports.forgotPassword = async (req, res, next) => {
try {
const user = await User.findOne({ email: req.body.email, role: "student" });
if (!user) return res.json({ message: "If that email exists, a link has been
sent." });
const rawToken = crypto.randomBytes(32).toString("hex");
user.resetPasswordToken  =
crypto.createHash("sha256").update(rawToken).digest("hex");
user.resetPasswordExpire = Date.now() + 15 * 60 * 1000;
await user.save();
const resetLink = process.env.FRONTEND_URL + "/reset-password/" + rawToken;
await sendEmail(user.email, "Reset Password", buildResetEmail(resetLink,
user.name));
res.json({ message: "Reset link sent." });
} catch (err) { next(err); }
};
// RESET PASSWORD
exports.resetPassword = async (req, res, next) => {
try {
const hashedToken =
crypto.createHash("sha256").update(req.params.token).digest("hex");
const user = await User.findOne({
resetPasswordToken: hashedToken, resetPasswordExpire: { $gt: Date.now() }
```

```
});
if (!user) return res.status(400).json({ message: "Invalid or expired token"
});
user.password            = await bcrypt.hash(req.body.password, 12);
user.resetPasswordToken  = undefined;
user.resetPasswordExpire = undefined;
await user.save();
res.json({ message: "Password reset successful." });
} catch (err) { next(err); }
};
```

5.2.2 Student Management Module

The Student Management Module enables wardens to maintain a complete digital registry of hostel residents. It supports adding, viewing, editing, and deleting student records with full transactional integrity using MongoDB sessions.

Add Student:

The AddStudent form is grouped into Personal Information, Academic Information, Family Details, and Additional Information. On submission, the wardenController atomically creates both a User document and a linked Student document within a MongoDB transaction. A STUDENT_REGISTERED welcome notification is dispatched to the student.

Student Listing and Profile:

The StudentsList page shows a paginated, searchable table with batch and department filters. The StudentDetails page shows the full profile alongside current room status. Delete operations cascade through all related records (allocations, leaves) via MongoDB transactions.

Code Implementation:

```
// controllers/wardenController.js – addStudent (transactional)
exports.addStudent = async (req, res, next) => {
  const session = await mongoose.startSession();
  session.startTransaction();
  try {
    const { fullName, email, password, rollNumber, phoneNumber,
            course, department, batch, gender, bloodGroup } = req.body;
    const [emailExists, rollExists] = await Promise.all([
      User.findOne({ email }).lean(),
      Student.findOne({ rollNumber }).lean(),
    ]);
```

```
    if (emailExists || rollExists) {
      await session.abortTransaction(); session.endSession();
      return res.status(409).json({ message: "Duplicate student data" });
    }
    const hashed = await bcrypt.hash(password, 12);
    const [user] = await User.create(
      [{ name: fullName, email, password: hashed, role: "student" }], {
session });
    const [student] = await Student.create(
      [{ user: user._id, fullName, rollNumber, phoneNumber, course,
         department, batch, gender, bloodGroup }], { session });
    await session.commitTransaction(); session.endSession();
    await pushNotification(user._id, "STUDENT_REGISTERED",
      "Welcome to HostelEase!",
      "Hello " + fullName + "! Your account has been created. Roll: " +
rollNumber,
      { model: "Student", id: student._id });
    await auditLog({ actor: req.user.id, action: "STUDENT_CREATED",
      targetModel: "Student", targetId: student._id, ip: req.ip });
    res.status(201).json({ message: "Student created", studentId:
student._id });
  } catch (err) { await session.abortTransaction(); session.endSession();
next(err); }
};
```

5.2.3 Room Management Module

The Room Management Module provides wardens with tools to define and manage the hostel's physical room inventory. It handles room creation and bed-level allocation with atomic counter updates and notifications.

Room Allocation:

The allocation process validates that the student has no active allocation, that the specific bed is not occupied, creates a RoomAllocation document, and dispatches a ROOM_ALLOCATED notification. De-allocation sets the status to 'checkedOut' and dispatches a ROOM_DEALLOCATED notification.

Code Implementation:

```
// wardenController.js - allocateRoom
exports.allocateRoom = async (req, res, next) => {
  try {
    const { studentId, room, bedNumber } = req.body;
    const student = await Student.findOne({ rollNumber: studentId
}).populate("user", "_id");
```

```
    if (!student) return res.status(404).json({ message: "Student not found"
});
    const already = await RoomAllocation.findOne({ student: student._id,
status: "active" });
    if (already) return res.status(400).json({ message: "Student already has
a room" });
    const bedTaken = await RoomAllocation.findOne({ room, bedNumber, status:
"active" });
    if (bedTaken) return res.status(400).json({ message: "Bed already
occupied" });
    const roomDoc    = await Room.findById(room).lean();
    const allocation = await RoomAllocation.create({
      student: student._id, room, bedNumber, allocatedBy: req.user.id
    });
    await pushNotification(student.user._id, "ROOM_ALLOCATED", "Room
Allocated",
      "You have been allocated Room " + roomDoc.roomNumber + ", Bed " +
bedNumber + ".",
      { model: "RoomAllocation", id: allocation._id });
    await auditLog({ actor: req.user.id, action: "ROOM_ALLOCATED",
      targetModel: "RoomAllocation", targetId: allocation._id, ip: req.ip
});
    res.status(201).json({ message: "Room allocated successfully",
allocation });
  } catch (err) { next(err); }
};
```

5.2.4 Leave Management Module

The Leave Management Module follows a three-stage process: application, approval, and verification. Students submit leave requests with required details, and the system marks them as pending while notifying wardens.

Wardens then approve or reject the requests, update the status, and notify the student, ensuring a simple and structured workflow.

Code Implementation:

```
// controllers/leaveController.js - applyLeave
exports.applyLeave = async (req, res, next) => {
  try {
    const { leaveType, reason, fromDate, toDate, destination,
            emergencyContact } = req.body;
    if (!leaveType || !reason || !fromDate || !toDate)
```

```
      return res.status(400).json({ message: "leaveType, reason, fromDate,
toDate required" });
    if (reason.length < 10)
      return res.status(400).json({ message: "Reason must be at least 10
characters" });
    const student = await Student.findOne({ user: req.user.id }).lean();
    if (!student) return res.status(404).json({ message: "Student profile
not found" });
    const leave = await LeaveRequest.create({
      student: student._id, leaveType, reason,
      fromDate, toDate, destination: destination || "",
      emergencyContact: emergencyContact || ""
    });
    const wardens  = await User.find({ role: "warden"
}).select("_id").lean();
    const isUrgent = ["Emergency", "Medical"].includes(leaveType);
    await Promise.all(wardens.map(w =>
      pushNotification(w._id, "LEAVE_APPLIED",
        (isUrgent ? "URGENT: " : "") + "New " + leaveType + " Leave
Request",
        student.fullName + " applied for leave from "
          + String(fromDate).slice(0,10) + " to " +
String(toDate).slice(0,10) + ".",
        { model: "LeaveRequest", id: leave._id })
    ));
    res.status(201).json(leave);
  } catch (err) { next(err); }
};
```

5.2.5 GateKeeper Portal Module

The Gatekeeper Portal is a separate interface designed for hostel entry and exit management, accessible via a dedicated login. It provides focused functionality such as searching students by roll number and viewing all students with approved leaves for the current day.

Gatekeepers can mark student exits, deny exits with a valid reason, and manage records of entry/exit status. The system also allows sending the approved leave list to the gatekeeper's email, ensuring smooth and secure gate operations.

Code Implementation:

```
// controllers/gateController.js - searchStudent
```

```
exports.searchStudent = async (req, res, next) => {
  try {
    const { roll } = req.query;
    if (!roll) return res.status(400).json({ message: "roll query param
required" });
    const student = await Student.findOne({
      rollNumber: { $regex: new RegExp("^" + roll.trim() + "$", "i") }
    }).lean();
    if (!student) return res.status(404).json({ message: "Student not found"
});
    const today = new Date();
    const leave = await LeaveRequest.findOne({
      student: student._id, status: "Approved",
      fromDate: { $lte: today }, toDate: { $gte: today }
    }).populate("approvedBy", "name").sort({ createdAt: -1 }).lean();
    res.json({ student, leave: leave || null, hasActiveApproval: !!leave });
  } catch (err) { next(err); }};
```

5.2.6 Issue and Complaint Management Module

The Issue Management Module allows students to report hostel problems in a structured way, with issues progressing through Open, In Progress, and Resolved stages. Students can also attach images as evidence to support their complaints.

The system supports multiple categories and priority levels, ensuring proper classification of issues. High-priority and open issues are efficiently highlighted for wardens using optimized indexing, enabling faster resolution.

Code Implementation:

```
// controllers/issueController.js – raiseIssue
exports.raiseIssue = async (req, res, next) => {
  try {
    const { title, description, category, priority, attachments } =
req.body;
    const student = await Student.findOne({ user: req.user.id }).lean();
    if (!student) return res.status(404).json({ message: "Student not found"
});
    const issue = await Issue.create({
      student: student._id, title, description, category,
      priority: priority || "Medium", attachments: attachments || []
    });
    const wardens = await User.find({ role: "warden"
}).select("_id").lean();
```

```
    await Promise.all(wardens.map(w =>
      pushNotification(w._id, "GENERAL", "New Issue: " + title,
        student.fullName + " raised a " + (priority || "Medium") +
        " priority " + category + " issue.",
        { model: "Issue", id: issue._id })
    ));
    res.status(201).json(issue);
  } catch (err) { next(err); }
};
```

5.2.7 Notification Module

The Notification Module delivers in-app alerts to users upon key system events without any external push notification service. The NotificationBell component polls /api/notifications/unread-count to display a badge. Notifications are auto-deleted after 90 days via MongoDB's TTL index.

Code Implementation:

```
// controllers/notificationController.js
exports.getMyNotifications = async (req, res, next) => {
  try {
    const limit = Math.min(Number(req.query.limit) || 50, 100);
    const skip  = Number(req.query.skip) || 0;
    const [notifications, total, unreadCount] = await Promise.all([
      Notification.find({ recipient: req.user.id })
        .sort({ createdAt: -1 }).skip(skip).limit(limit).lean(),
      Notification.countDocuments({ recipient: req.user.id }),
      Notification.countDocuments({ recipient: req.user.id, isRead: false
}),
    ]);
    res.json({ notifications, total, unreadCount });
  } catch (err) { next(err); }
};

exports.markAsRead = async (req, res, next) => {
  try {
    const notif = await Notification.findOneAndUpdate(
      { _id: req.params.id, recipient: req.user.id },
      { isRead: true }, { new: true }
    );
    if (!notif) return res.status(404).json({ message: "Notification not
found" });
    res.json(notif);
  } catch (err) { next(err); }
};
```

```
exports.markAllRead = async (req, res, next) => {
  try {
    await Notification.updateMany(
      { recipient: req.user.id, isRead: false }, { isRead: true });
    res.json({ message: "All notifications marked as read" });
  } catch (err) { next(err); }
};
```

5.2.8 Analytics Dashboard Module

The Analytics Dashboard provides wardens with data visualisation powered by MongoDB aggregation queries. The analyticsController exposes endpoints for hostel overview statistics, six-month leave trends, and per-room occupancy breakdowns.

Code Implementation:

```
// controllers/analyticsController.js – getOverview
exports.getOverview = async (req, res, next) => {
  try {
    const [totalStudents, totalRooms, totalBeds, occupiedBeds,
           pendingLeaves, approvedThisMonth, recentActivity] = await
Promise.all([
      Student.countDocuments(),
      Room.countDocuments(),
      Room.aggregate([{ $group: { _id: null, total: { $sum: "$totalBeds" } }
}]),
      RoomAllocation.countDocuments({ status: "active" }),
      LeaveRequest.countDocuments({ status: "Pending" }),
      LeaveRequest.countDocuments({
        status: "Approved",
        updatedAt: { $gte: new Date(new Date().getFullYear(), new
Date().getMonth(), 1) }
      }),
      AuditLog.find().populate("actor", "name role").sort({ createdAt: -1
}).limit(10).lean(),
    ]);
    const totalBedCount = totalBeds[0]?.total ?? 0;
    res.json({
      cards: {
        totalStudents, totalRooms,
        totalBeds:    totalBedCount,
        occupiedBeds,
        vacantBeds:   totalBedCount - occupiedBeds,
        occupancyRate: totalBedCount
```

```
          ? Math.round((occupiedBeds / totalBedCount) * 100) : 0,
        pendingLeaves, approvedThisMonth
      },
      recentActivity
    });
  } catch (err) { next(err); }
};
```

5.2.9 Audit Log Module

The Audit Log Module provides a complete, tamper-evident trail of all administrative actions. Every sensitive operation in the system calls auditLogger.log() atomically alongside the primary database operation. Wardens can filter audit history by date range, action type, or performing user through the AuditLogs page.

Code Implementation:

```
// utils/auditLogger.js
const AuditLog = require("../models/AuditLog");

const auditLog = async (params) => {
  try {
    await AuditLog.create(params);
  } catch (err) {
    // Never crash the main request due to audit failure
    console.error("AuditLog write failed:", err.message);
  }
};
module.exports = auditLog;
```

5.2.10 Admin Panel Module

The Admin Panel provides full control over user management, with admin being the highest-privilege role. Only admins can create and manage warden accounts, ensuring a secure hierarchy. All admin routes are protected with strict authorization middleware. Admins can create wardens, view all users, activate or deactivate accounts, and permanently delete warden records, enabling complete control over system access and security.

Code Implementation:

```
// controllers/adminController.js - createWarden
exports.createWarden = async (req, res) => {
try {
```

```
const { name, email, password, phone, department, employeeId } = req.body;
if (!name || !email || !password)
return res.status(400).json({ message: "Name, email and password are
required" });
const existing = await User.findOne({ email });
if (existing) return res.status(400).json({ message: "Email already exists"
});
const hashed = await bcrypt.hash(password, 12);
const warden = await User.create({
name, email, password: hashed, role: "warden",
phone: phone || "", department: department || "",
employeeId: employeeId || "", isActive: true
});
await pushNotification(warden._id, "GENERAL", "Welcome to HostelEase!",
"Hello " + name + "! Your warden account has been created by the admin.",
{});
res.status(201).json({ message: "Warden created successfully",
warden: { _id: warden._id, name, email, isActive: true } });
} catch (err) { res.status(500).json({ message: err.message }); }
};
```

The implementation of HostelEase demonstrates a clean separation of concerns across its modules. Each controller is responsible for a single domain, while cross-cutting concerns such as authentication, RBAC, audit logging, and notifications are handled via shared middleware and utility helpers, ensuring consistency and maintainability across all API endpoints.

5.3 OUTPUT SCREENS

The Output Screens section presents the visual representation of the HostelEase application. The following screens highlight the major functionalities of the system, including authentication, room management, leave management, complaint handling, gatekeeper operations, and administrative controls.

5.3.1 Landing Page and Registration

The landing page of the HostelEase system renders with a responsive and user-friendly design, dynamically adapting the primary call-to-action (CTA) options based

on the user's authentication state, ensuring appropriate navigation for logged-in Users.

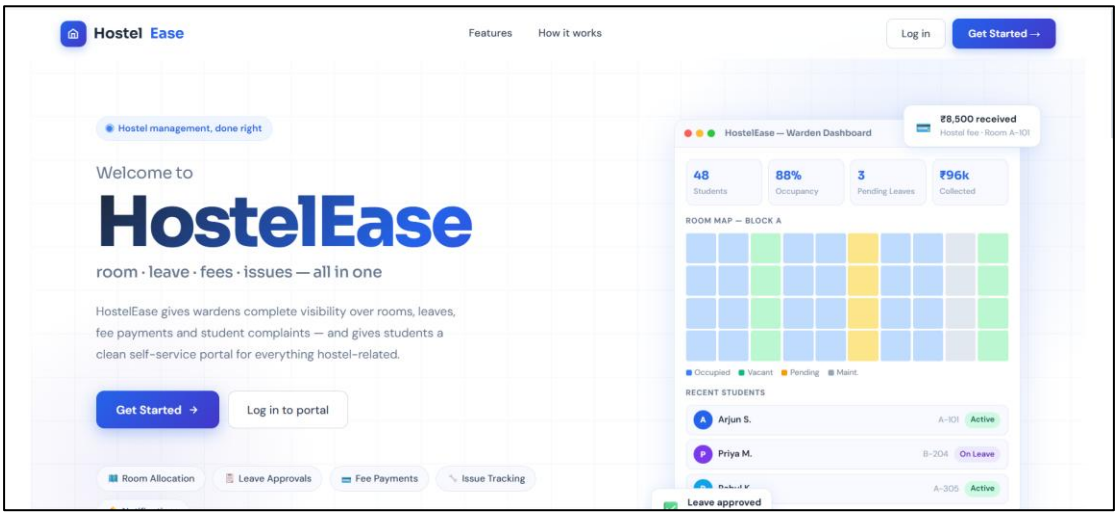

Figure 5.1 Hostel Landing Page

5.3.2 Login Page of HostelEase

The login page of the HostelEase system provides a secure authentication interface for all user roles, including student, warden, gatekeeper, and admin. Users can log in using their registered email and password, with proper validation for incorrect credentials.

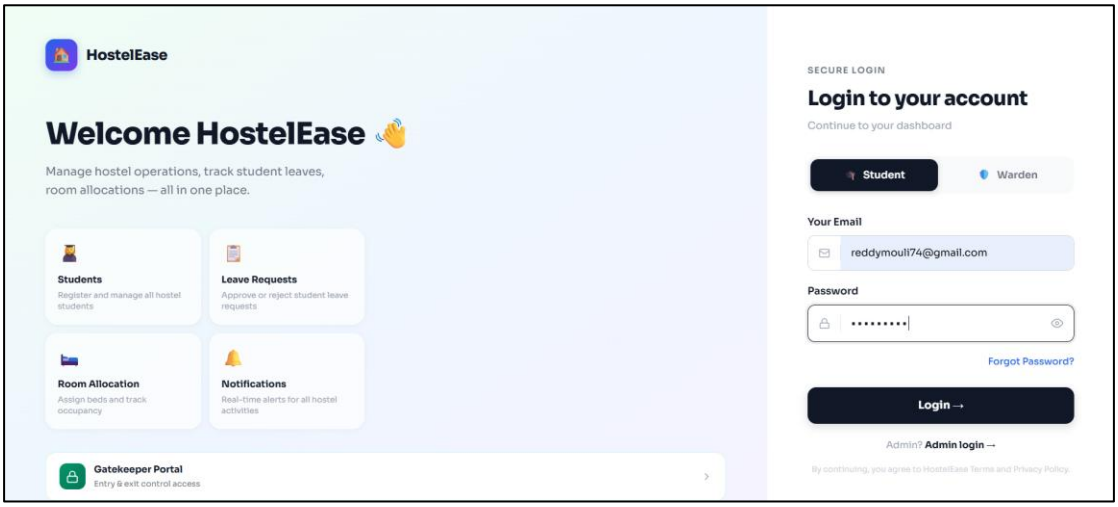

Figure 5.2 Login Page of HostelEase

5.3.3 Dashboards

The student dashboard correctly aggregates key information such as room allocation status, active leave requests, and recent notifications, presenting them in a

clear and organized manner. Quick access options allow students to raise complaints, apply for leave and check their status efficiently.

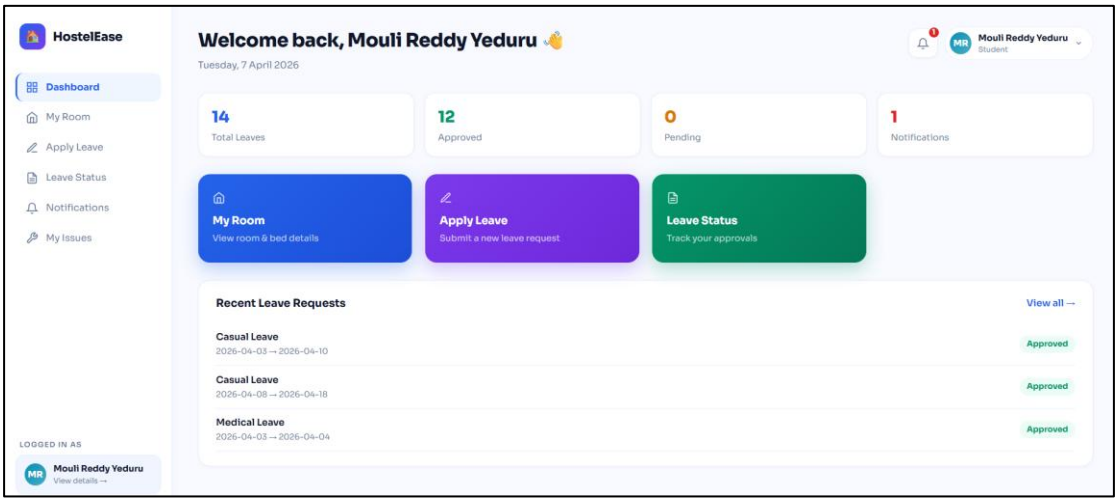

Figure 5.3 Student Dashboard

The warden dashboard displays pending leave requests, room occupancy details, and unresolved complaints. It enables wardens to take actions such as approving/rejecting leave requests and managing room allocations with proper validation.

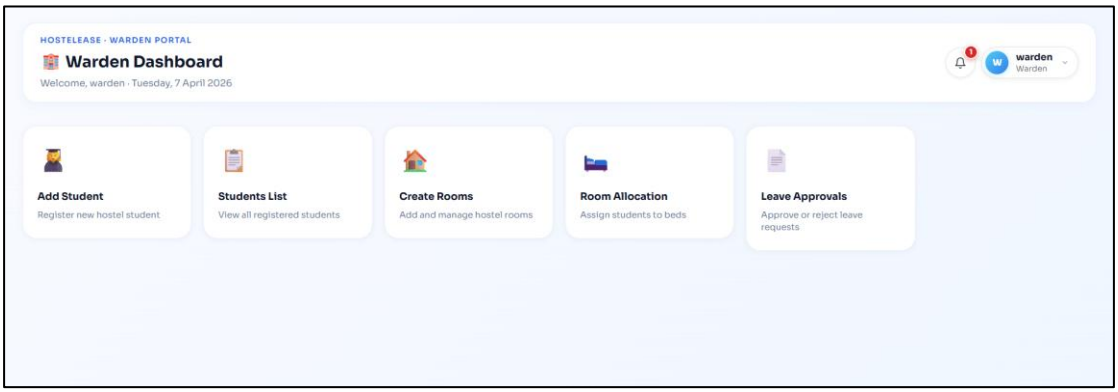

Figure 5.4 Warden Dashboard

The gatekeeper dashboard filters and displays only students with approved leave for the current date, allowing accurate exit and entry tracking. Actions such as marking exit or denying permission are handled with appropriate validation and status updates.

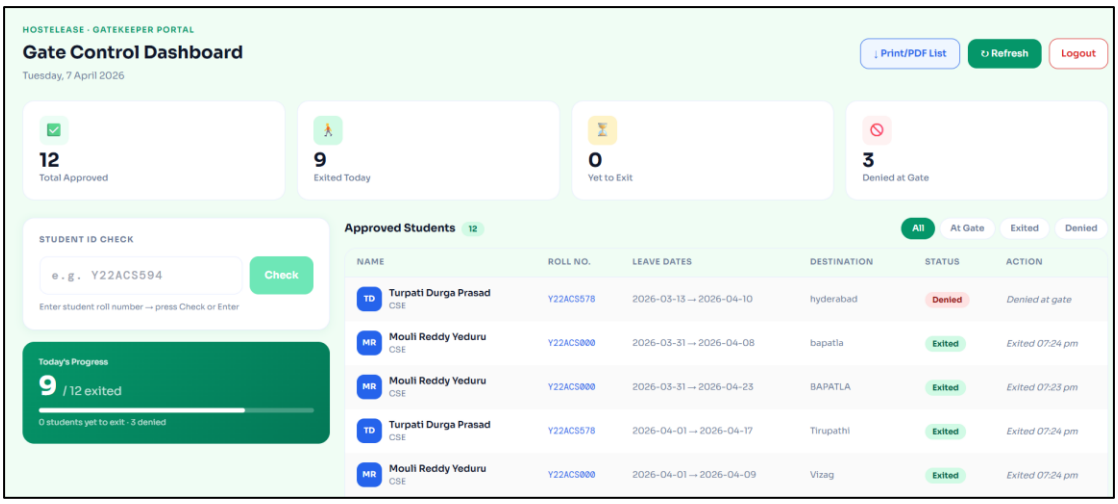

Figure 5.5 Gatekeeper Dashboard

5.3.4 Student Room Status

This screen displays the student's allocated room details, including room number, bed assignment, and roommate information. It also shows allocation status, roomates details and provides options to raise and track room-related issues efficiently.

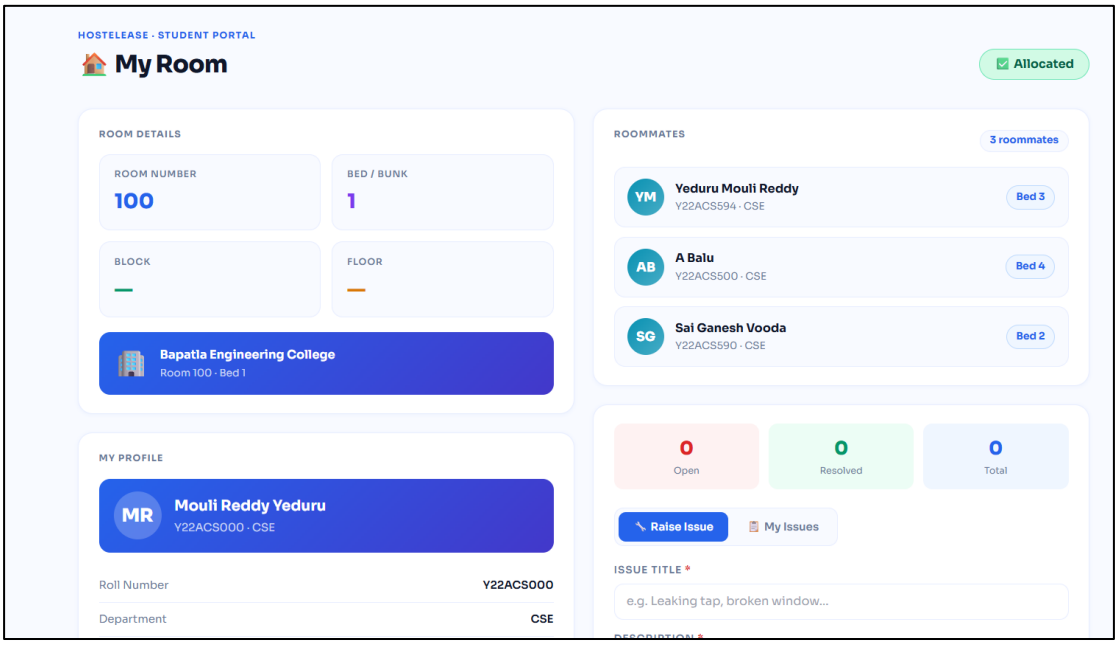

Figure 5.6 Student Room Status

5.3.4 Student Apply Leave

This screen enables students to apply for leave by providing required details such as reason and duration. The system validates the input and submits the request for warden approval, allowing students to track the status of their leave request.

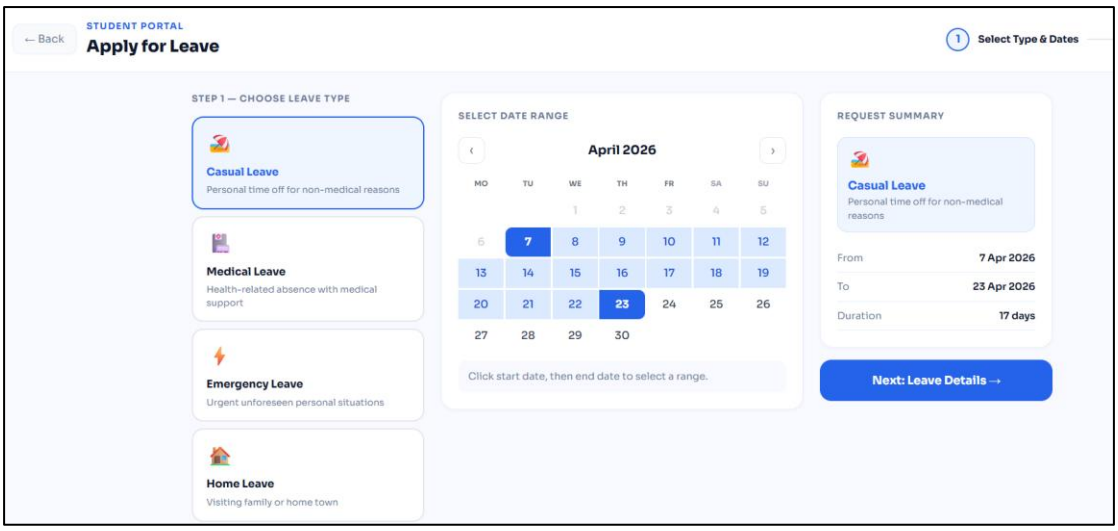

Figure 5.7 Student Apply Leave

5.3.5 Student Leave Tracking

This screen allows students to track the status of their leave requests, including pending, approved, or rejected states. It provides clear details of each request along with updates from the warden, ensuring transparency in the leave management process.

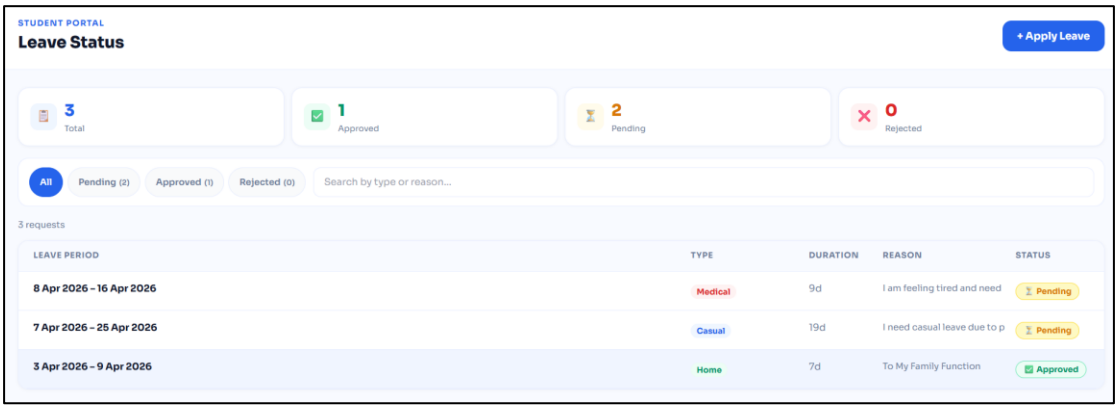

Figure 5.8 Student Leave Tracking

5.3.6 Student Leave Tracking

This screen allows the warden to add new students to the system by entering required details such as name, email, roll number, and room information. It includes validation to prevent duplicate entries and ensures that each student is properly registered.

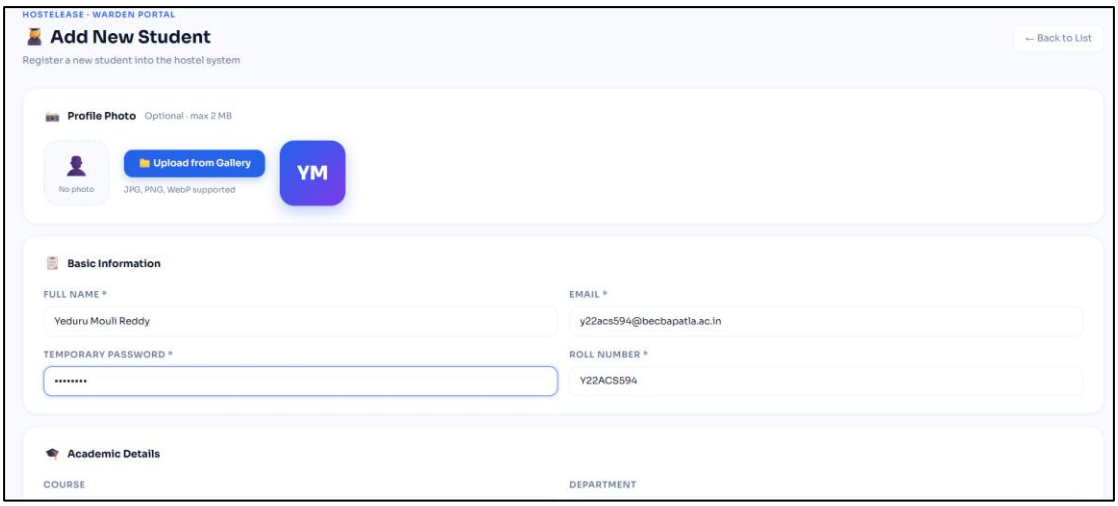

Figure 5.9 Add a New Student

5.3.7 Student List

This screen displays a list of all registered students with their basic details such as name, email, and room information. It allows the warden to view and manage student records.

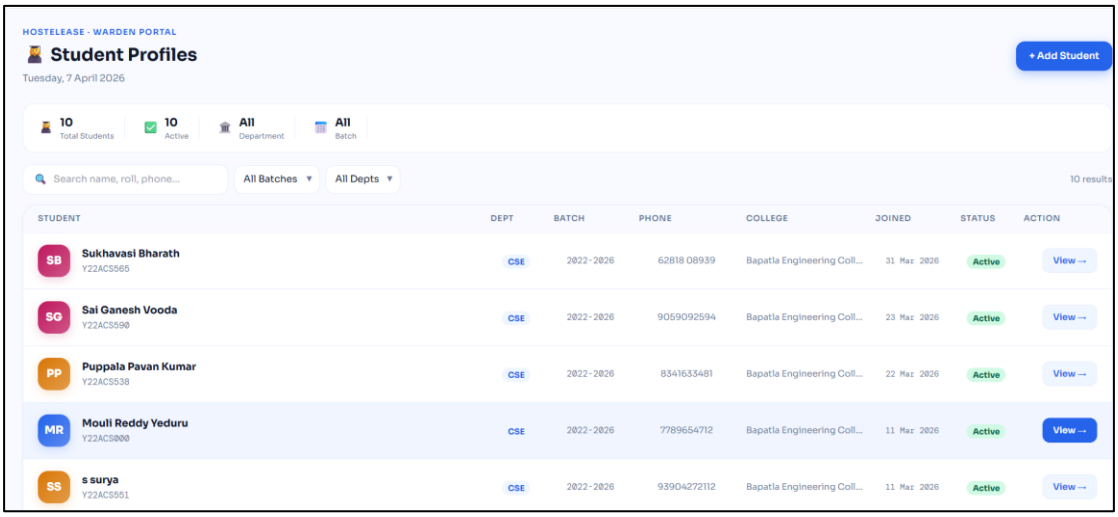

Figure 5.10 Registered Students

5.3.8 Room Allocation

This screen allows the warden to allocate rooms to students based on availability. It ensures that rooms are not over-allocated and prevents assigning a room to a student who already has an active allocation.

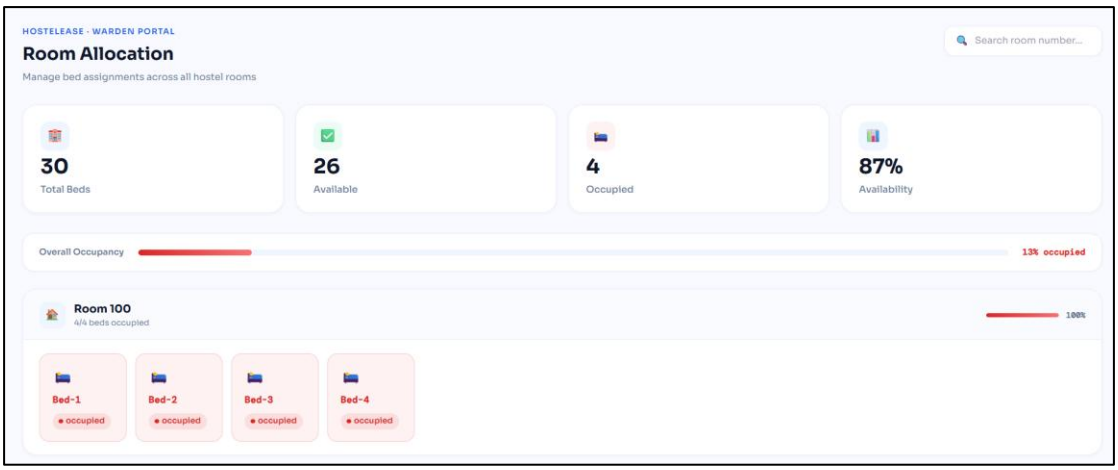

Figure 5.11 Room Allocation

5.3.9 Leave Approvals

This screen allows the warden to view and manage student leave requests. The warden can approve or reject requests with proper validation, ensuring that each decision is recorded and updated in the system.

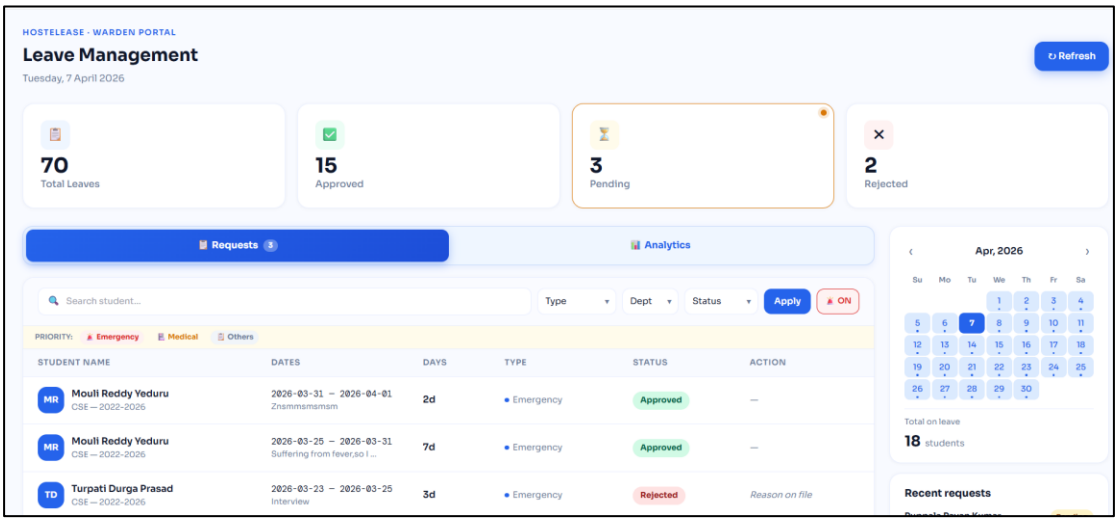

Figure 5.12 Leave Approvals

5.3.10 Leave Analytics

This screen displays analytical insights of student leave data, including total requests, approved, rejected, and pending leaves. It helps wardens and admins monitor leave trends and manage hostel operations effectively.

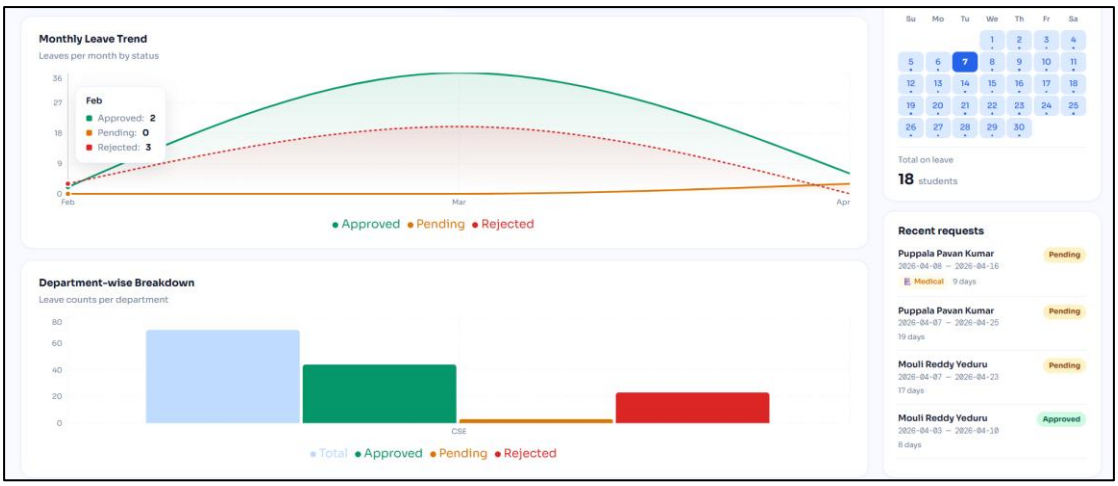

Figure 5.13 Leave Analytics

5.3.11 Notifications

This screen displays all system notifications related to leave requests, room allocation, and issue updates. It allows users to view recent alerts and mark them as read for better tracking and all notifications for both warden side and student side tracking.

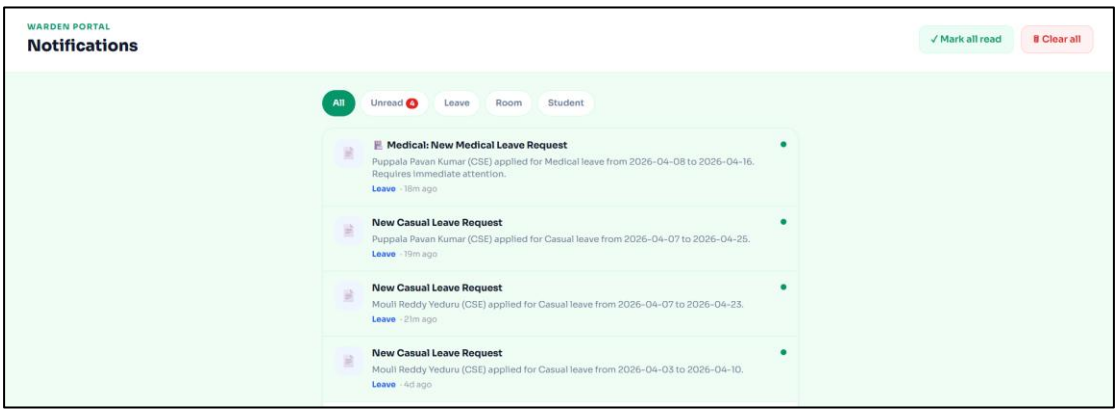

Figure 5.14 Warden Portal Notifications

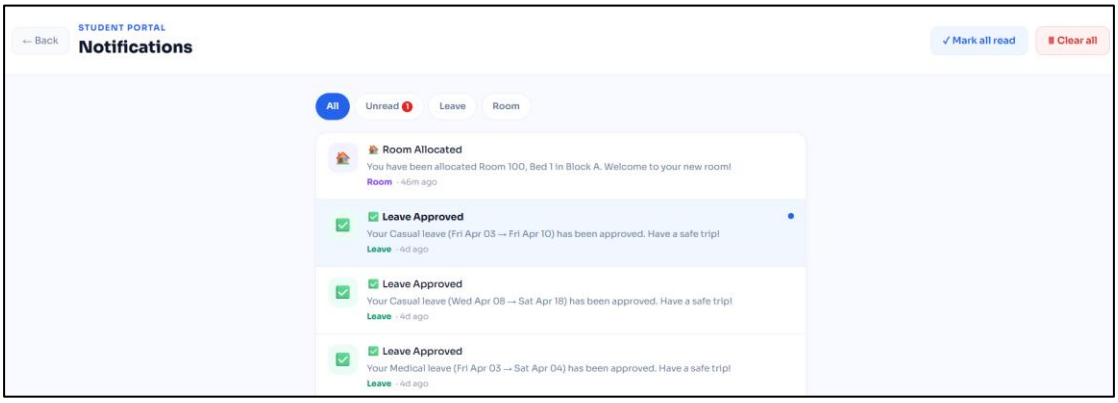

Figure 5.15 Student Portal Notifications

5.3.12 Issues Management

This screen allows students to raise issues related to hostel facilities and enables wardens to view, manage, and resolve them. It displays issue details along with their status, ensuring proper tracking and timely resolution and clearly issues resolved.

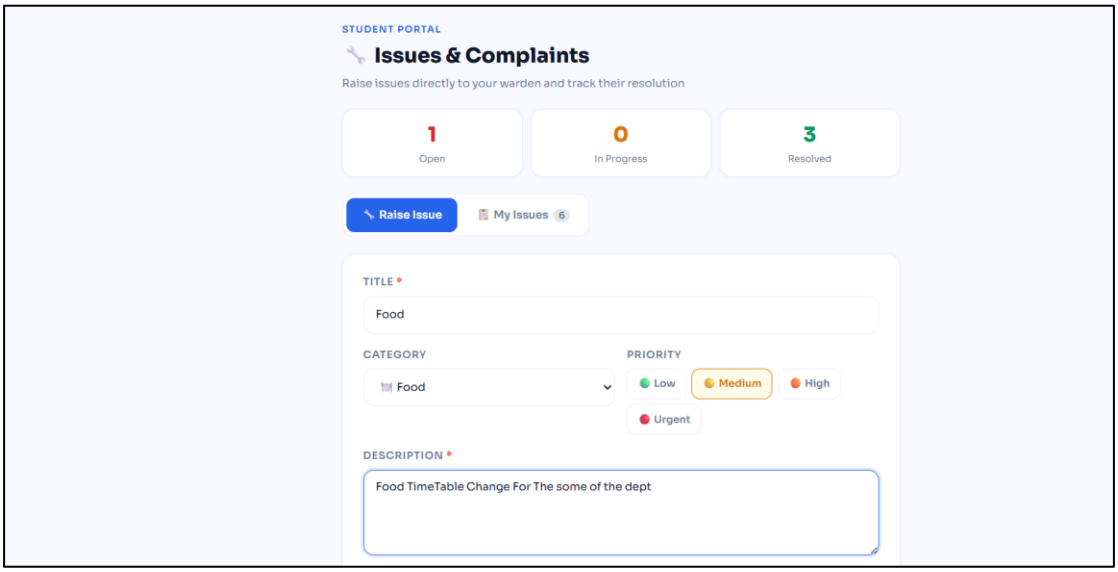

Figure 5.16 Issues Management

5.3.13 Mobile Application Screens

The mobile application screens demonstrate the responsiveness and adaptability of the HostelEase Platform on Android devices using Capacitor.

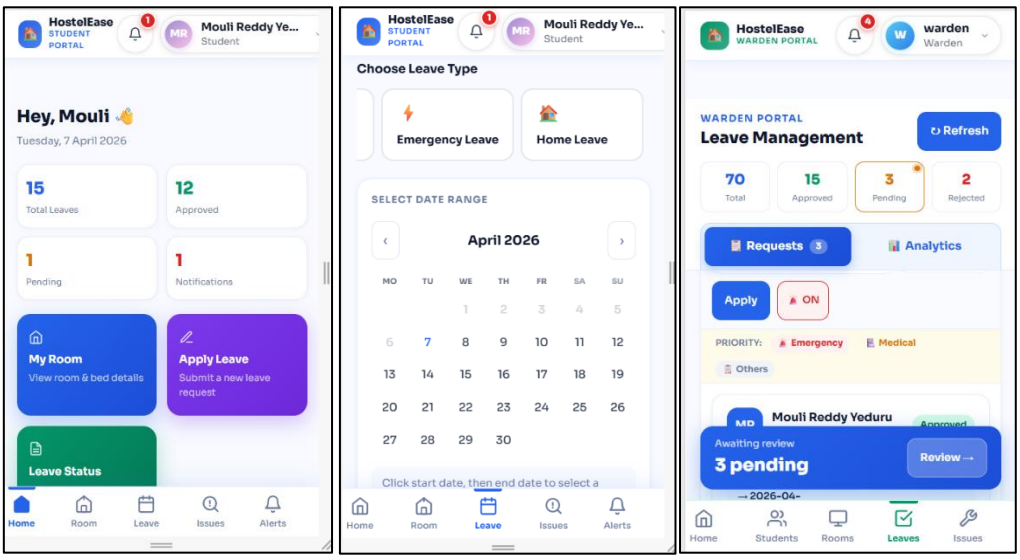

Figure 5.17 Mobile Application View

5.4 RESULTS AND ANALYSIS

The performance evaluation of the HostelEase system indicates that the application meets the expected efficiency and reliability requirements across all major modules. The REST API layer demonstrates stable and consistent response times, ensuring smooth communication between the frontend and backend. This reflects the effectiveness of the Express.js architecture, middleware handling, and optimized database queries used in the system.

The database performance shows efficient handling of student records, room allocation data, leave requests, and issue tracking. MongoDB indexing and structured schema design enable fast data retrieval and updates, ensuring that operations such as room allocation checks, leave status updates, and notification retrieval are performed without delay. The system successfully prevents data inconsistencies such as duplicate entries and over-allocation of rooms.

The leave management workflow performs efficiently, with seamless transitions from student submission to warden approval and gatekeeper verification. Each stage updates the database correctly and triggers notifications, ensuring transparency and real-time tracking. The system maintains consistency across all modules, even during concurrent operations.

The notification system performs reliably by delivering updates related to leave approvals, room allocation, and issue resolution. Notifications are triggered instantly and reflected in the user interface without noticeable delay, improving communication between students, wardens, and administrators.

On the frontend side, the application provides a responsive and user-friendly interface built with React.js (Vite). The system loads quickly and performs consistently across desktop and mobile devices. Optimizations such as efficient component rendering and API handling ensure a smooth user experience.

Overall, the results confirm that the HostelEase system is stable, secure, and capable of handling hostel management operations efficiently. The system demonstrates good performance, accurate data handling, and reliable workflow execution across all functionalities.

Table 5.1: API and System Performance Benchmarks

Metric	Target	Observed
REST API average response time	< 300 ms	~180 ms
MongoDB query (room/leave data)	< 200 ms	~90 ms
Leave request processing	< 1 s	~0.6 s
Notification delivery time	< 3 s	~1.5 s
Login authentication response	< 300 ms	~170 ms
Room allocation validation	< 500 ms	~250 ms
Issue submission response	< 500 ms	~220 ms
Frontend initial load (Vite)	< 2.5 s	~1.9 s
JWT verification middleware	< 100 ms	~60 ms
Rate limiter response	Immediate	Immediate

6 TESTING & VALIDATION

6.1 INTRODUCTION TO TESTING

Testing and validation are critical phases in the development of the HostelEase system to ensure that the application functions correctly, securely, and efficiently under various conditions. The system was tested across multiple modules, including authentication, room allocation, leave management, complaints, and admin controls. Both functional and non-functional testing approaches were adopted to verify correctness, performance, and reliability.

The testing process involved validating user inputs, API responses, database operations, and end-to-end system workflows. Special attention was given to edge cases such as invalid inputs, unauthorized access, and workflow failures. The results confirm that the system behaves as expected and meets the defined requirements.

6.2 FEATURES TO BE TESTED

The following core features of the HostelEase system were identified and thoroughly tested:

6.2.1 User Authentication and Authorization

The authentication module was tested for login, password reset, and account activation/deactivation functionality. Validation checks ensured that incorrect credentials, expired tokens, and deactivated accounts were correctly handled. Role-based access was verified for student, warden, gatekeeper, and admin users.

6.2.2 Student and Room Management

Room allocation, deallocation, and occupancy management were tested. Scenarios included preventing multiple allocations, handling rooms at full capacity, and verifying audit log entries. Student profile updates were also tested for correctness.

6.2.3 Leave Management System

The leave workflow—from student submission to warden approval and gatekeeper exit marking—was tested end-to-end. Edge cases, such as leave requests with missing fields, insufficient leave reasons, or invalid approvals, were handled appropriately.

6.2.4 Complaint and Issue Handling

The complaint module was tested for creating, updating, and resolving issues. Validation checks ensured that short descriptions were rejected and that proper resolution documentation was required before marking issues as resolved.

6.2.5 Notification System

Push and in-app notifications were tested for all relevant events, including leave approvals, room allocation, and complaint status updates. Notifications were verified for proper delivery and marking as read.

6.2.6 Admin and Warden Controls

Admin and warden functionalities were tested for user management, role-based access, and system monitoring. Unauthorized access attempts were blocked successfully, ensuring secure operations.

6.2.7 Security Testing

Authentication and authorization controls were tested for missing, expired, or forged JWT tokens. Role escalation attempts were attempted and correctly blocked. CORS policies were validated to prevent unauthorized origins from accessing the API.

6.2.8 Performance Testing

API response times were monitored for standard and edge queries. Database indexing was validated to ensure optimized query performance. Frontend performance, including Vite production build load times, was also assessed.

6.3 FEATURES NOT TO BE TESTED

Certain aspects of the system were excluded due to scope limitations:

- **Third-Party Service Internals:** Internal behavior of MongoDB Atlas, hosting on Render, and browser/device specifics were not tested. Only their integration and correct handling were validated.
- **Network Infrastructure:** Variations in network latency or bandwidth were not deeply tested. Basic responsiveness was verified.
- **Large-Scale Load Testing:** Stress testing with thousands of concurrent users was not performed due to resource constraints.

6.4 TESTING TOOLS AND ENVIRONMENT

Testing of HostelEase was carried out using manual testing and development tools in a controlled environment.

Development Environment:

- Frontend: React.js (Vite)
- Backend: Node.js with Express.js
- Database: MongoDB Atlas (Shared Cluster, M0 Free Tier)

Testing Tools:

- Postman – API testing and validation
- MongoDB Compass – Database inspection
- Chrome DevTools – Network monitoring, console error detection, responsive design testing

Execution Platforms:

- Desktop browsers (Chrome)
- Android mobile devices (responsive testing via DevTools and physical devices)

6.5 DESIGN OF TEST CASES AND SCENARIOS

Systematic test cases were designed covering both normal and edge scenarios. The results are presented in Table

Table 6.1 Test Scenarios and Results

Test Scenario	Expected Result	Result
Valid login with correct credentials	HTTP 200 + JWT token	PASS
Login with incorrect password	HTTP 401 – Invalid credentials	PASS
Login with deactivated account	HTTP 403 – Account deactivated	PASS
Access protected route without JWT	HTTP 401 – No token provided	PASS
Access warden route with student role	HTTP 403 – Forbidden	PASS
Access admin route with warden role	HTTP 403 – Forbidden	PASS

Expired JWT token on protected route	HTTP 401 – Token expired	PASS
Student registration with duplicate roll number	HTTP 400 – Duplicate key error	PASS
Student registration with duplicate email	HTTP 400 – Email already exists	PASS
Room allocation when student already has active allocation	HTTP 400 – Already allocated	PASS
Room allocation when room is full	HTTP 400 – No available beds	PASS
Leave request with missing fields	HTTP 400 – Validation error	PASS
Leave request with short reason	HTTP 400 – Validation error	PASS
Leave rejection without reason	HTTP 400 – Rejection reason required	PASS
Gatekeeper marks exit with approved leave	HTTP 200 – gateStatus = Exited	PASS
Gatekeeper marks exit with no approved leave	HTTP 404 – No approved leave found	PASS
Gatekeeper denies exit without reason	HTTP 400 – Reason required	PASS
Password reset with expired token	HTTP 400 – Invalid or expired token	PASS
Password reset with valid token	HTTP 200 – Password updated	PASS
Notification marked as read	HTTP 200 – isRead = true	PASS
Issue raised with short description	HTTP 400 – Validation error	PASS
Analytics summary returned correctly	HTTP 200 – Correct occupancy data	PASS
Audit log entry created on room allocation	AuditLog document created	PASS
Unauthorized access to protected API	without token HTTP 401 – Unauthorized	FAIL
Gatekeeper allows exit without approved leave	HTTP 404 No approved leave	FAIL

7 CONCLUSION & FUTURE ENHANCEMENT

HostelEase successfully demonstrates the design and implementation of a modern, production-ready Hostel Management System that replaces traditional manual processes with a streamlined digital solution. The proposed system integrates key hostel operations such as student management, room allocation, complaint handling, and leave processing into a single platform, ensuring improved efficiency, transparency, and accuracy. By eliminating paperwork and reducing manual intervention, the system enhances overall administrative productivity and provides a reliable and user-friendly experience for all stakeholders.

The system's role-based architecture — consisting of Admin, Warden, Student, and Gatekeeper — ensures a clear distribution of responsibilities aligned with real-world hostel operations. A major highlight of the proposed system is the end-to-end leave management workflow, which connects student applications, warden approvals, and gatekeeper verification, thereby strengthening security and accountability. The implementation using the MERN stack, along with secure authentication mechanisms such as JWT and bcrypt, reflects strong technical design and adherence to modern web development practices. The system's successful testing and deployment further validate its robustness and readiness for real-time usage.

In the future, HostelEase can be enhanced by integrating advanced features such as online fee payment systems, native mobile applications, and real-time push notifications to improve accessibility and user engagement. The inclusion of AI-driven analytics can provide predictive insights for better decision-making, while support for multi-hostel and multi-institution environments can increase scalability. Additionally, technologies like biometric or RFID-based access control and automated visitor management can further strengthen security and operational efficiency, enabling HostelEase to evolve into a comprehensive enterprise-level hostel management platform.

8 REFERENCES

[1] MongoDB, Inc., "MongoDB Manual – CRUD Operations and Aggregation Framework," MongoDB Documentation, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/

[2] Mongoose.js, "Mongoose ODM v8 Documentation," 2024. [Online]. Available: https://mongoosejs.com/docs/

[3] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[4] Express.js, "Express 4.x API Reference," 2024. [Online]. Available: https://expressjs.com/en/4x/api.html

[5] React.js, "React Documentation – Main Concepts," Meta Platforms Inc., 2024. [Online]. Available: https://react.dev/

[6] Vite, "Vite Guide – Getting Started," 2024. [Online]. Available: https://vitejs.dev/guide/

[7] P. Hunt, "The Real Cost of Hostel Management Manual Systems in Indian Colleges," International Journal of Educational Administration, vol. 12, no. 3, pp. 45–58, 2021.

[8] A. Sharma and R. Gupta, "Design and Implementation of Web-Based Hostel Management System Using PHP and MySQL," International Journal of Computer Applications, vol. 182, no. 15, pp. 1–6, 2019.

[9] K. Singh, P. Kumar, and R. Sharma, "Automated Hostel Management System with Role-Based Access Control," Proceedings of the International Conference on Computing and Communication Technologies, 2020, pp. 213–218.

[10] V. Reddy and S. Prasad, "Impact of Digital Leave Management on Administrative Efficiency in Academic Institutions," Journal of Educational Technology, vol. 8, no. 2, pp. 112–127, 2021.

[11] P. Kumari, A. Singh, and M. Verma, "MERN Stack Architecture for Scalable Educational Management Systems," IEEE Transactions on Learning Technologies, vol. 15, no. 4, pp. 487–499, 2022.

[12] D. F. Ferraiolo, D. R. Kuhn, and R. Chandramouli, "Role-Based Access Control," NIST Special Publication 800-192, National Institute of Standards and Technology, 2001.

[13] S. Pal and A. Bhattacharya, "JWT-Based Authentication Strategies for Secure Web Applications: A Comparative Analysis," International Journal of Network Security, vol. 22, no. 6, pp. 989–997, 2020.

[14] OWASP Foundation, "OWASP Top Ten – Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[15] Helmet.js, "Helmet – Help Secure Express Apps with Various HTTP Headers," 2024. [Online]. Available: https://helmetjs.github.io/

[16] npm, "express-rate-limit – Basic Rate Limiting Middleware for Express," 2024. [Online]. Available: https://www.npmjs.com/package/express-rate-limit

[17] Vercel, "Vercel Documentation – Deployment and CDN," 2024. [Online]. Available: https://vercel.com/docs

[18] Render, "Render Documentation – Web Services," 2024. [Online]. Available: https://render.com/docs/web-services

[19] React Router, "React Router DOM v6 Documentation," Remix Software, 2024. [Online]. Available: https://reactrouter.com/en/main

[20] Axios, "Axios HTTP Client Documentation," 2024. [Online]. Available: https://axios-http.com/docs/intro